

成本不對稱下以品質為約束的 可恢復階層式 KV State 管理

Cost-Asymmetric Hierarchical KV State Management
for Long-Context LLM Inference

(作者資訊於投稿時填入)

Abstract

長上下文 (long-context) LLM 推論的核心系統瓶頸之一，是 KV cache 隨上下文長度線性成長並耗盡 GPU 高頻寬記憶體 (HBM)。既有工作已分別處理了這個問題的各個面向：可恢復的驅逐、學習式的未來效用預測、重算與傳輸的取舍，以及以品質為約束的壓縮。然而，據我們所知，尚無工作將這些決策整合到單一的線上決策之中。

本文的出發點是一個橫跨硬體的量化事實：重算與傳輸的成本比 κ 隨平台變動達 32 倍——於 AMD Instinct MI300X (192 GB HBM3) 為 6–21 倍，於 NVIDIA RTX 3090 (24 GB GDDR6X) 則為 54–190 倍。此差異使「單一套 KV 放置策略適用於所有硬體」的隱含前提不成立。

本文全程採單卡、單請求、上下文遞增的壓力軸，加速器決定的是這條軸能推多遠，而非換一個問題。在 24 GB 級的卡上，可推的距離由權重精度決定：BF16 權重僅餘 41,648 token 的 KV 預算，改用 AWQ-INT4 後同一張卡升至 120,320–547,744 token，相差 13.2 $\times$ 。在 192 GB 級的卡上，同一條軸延伸至更大的模型與 1M 級上下文。兩端皆為真實的部署情境，而最佳策略在二者上並不相同。

基於此，我們將 KV 放置形式化為一個受品質約束的序列決策問題，其動作候選集合包含 GPU 全精度、GPU FP8、GPU INT4、CPU、SSD 與「丟棄後重算」六種狀態，並在「品質退化不超過 ϵ 」的硬性約束下，聯合最佳化未來效用、傳輸成本、重算成本與記憶體壓力。量測顯示有效子集隨平台與上下文長度而變：於 sm₈₆，三個低精度階中僅 INT8 一階在檢索任務上保住品質 (95%，而 FP8 為 5%、INT4 為 0%)；SSD 與 Drop 的優劣次序在成本交叉點 P^* 處反轉，而 P^* 在同一台機器上隨模型剖面即變動 3.5 $\times$ 。「動作空間的有效子集可事先算出」本身即為本文的主張。我們特別論證：重算的價值不在於單次事件的成本（其代價比傳輸高 6–21 倍於 MI300X，54–190 倍於 RTX 3090），而在於它在被需要之前不佔用任何資源；其正確性因而完全取決於未來效用預測的品質。

我們在 RTX 3090 上量測了離線最優 oracle 相對於最佳可部署 baseline 的改善空間，並依預先訂定的判準給出條件式 GO：於生產 trace (Mooncake，中位長度 6,352 token) 為 8.4–9.2%，而於目標長度區間 (65K–131K) 升至 31.8–33.5%。兩個條件皆可由成本常數事先算出——權重須量化，且請求長度須落在 2–3.5 P^* 。我們另指出一項既有 baseline 未納入設計的約束：磁碟階的寫入頻寬。成本感知放置所需的持續寫入頻寬僅為逐層下推策略的 1/4 至 1/7。本文為初稿：第 6 節列出已完成的量測，其中數項推翻了本文原先的前提；線上策略尚未實作，尚未執行的實驗設計列於附錄 B。

Keywords: LLM inference; KV cache; memory hierarchy; long context; quality-constrained optimization; cost-sensitive learning; AMD MI300X; NVIDIA RTX 3090; vLLM

1 Introduction

Transformer 模型在自回歸生成時，會將每個已處理 token 的 key 與 value 向量保存下來，避免在生成每個新 token 時重算整段前綴。這組中間狀態即 KV cache，是以記憶體換取計算時間的經典設計。其代價是記憶體佔用隨上下文長度線性成長：對 Llama-3.1-8B (32 層、8 個 KV head、head dimension 128、BF16)，每個 token 需要 128 KiB，1M token 的上下文即需要 122 GB。在 80 GB 級的加速器上，這使得系統的限制因素從計算能力轉移到記憶體容量。

圍繞此問題的研究已密集展開。既有工作大致沿四條軸線展開：(i) 分層儲存，將 KV 卸載到 CPU DRAM 或 SSD [17, 25, 28, 37, 39]；(ii) 可恢復驅逐，使被降級的 KV 在重新變得重要時能夠召回 [10, 19]；(iii) 學習式未來效用預測，以訓練得到的策略取代手寫啟發式規則 [4, 11, 34]；(iv) 重算與傳輸的取舍 [18, 21, 23]。此外，KVserve [30] 首次將模型品質寫為壓縮決策的硬性約束，而 Fang 等人 [14] 首次將分層 KV 放置形式化並推導理論上界。

本文的出發點：一個不再成立的前提。 上述工作幾乎全部在 NVIDIA A100/H100 級硬體 (40–80 GB HBM) 上評估。我們的量化分析 (第 2 節) 顯示，在 AMD Instinct MI300X (192 GB HBM3) 上，8B–14B 級模型於 128K 上下文下的單請求 KV footprint 僅 15.6–19.5 GB，佔 HBM 不到 10%。在此類硬體上，既有文獻用以立論的「單請求容量壓力」場景幾乎不存在。

這並不代表問題消失，而是同一條壓力軸上的位置不同。KV 佔用隨上下文線性成長，故任何容量都有耗盡的長度；192 GB 只是把那個長度往後推。Llama-3.1-8B 於 1M token 上下文需 122 GB KV，單張 MI300X 恰可容納而 H100 不能——壓力軸不變，可達的區間變了。本文因此在兩張卡上量同一件事：單請求、上下文遞增、直到 KV 耗盡。多租戶下的機會成本是另一個問題，本文不處理 (第 7 節)。

第二個觀察：卸載在大容量 HBM 硬體上更昂貴。 MI300X 的 HBM 頻寬 (5.3 TB/s 峰值) 與主機鏈路頻寬 (PCIe Gen5 $\times$ 16，單向理論 63.0 GB/s) 之比為 84:1，而 H100 為 52:1，MI355X 更達 127:1。此比值持續惡化，因

為 HBM 容量與頻寬逐代成長，而主機鏈路自 MI300X 至 MI355X 完全未變。灌滿或抽乾 MI300X 上 Llama-3.1-8B 的 157.8 GiB KV 預算，經 PCIe 需 2.69 秒，而在 HBM 內僅需 32 毫秒。此比值必然等於頻寬比 84 倍。

貢獻。

1. 我們量化了重算對傳輸的成本比 κ 隨平台變動達 32 倍（MI300X 上 6–21 倍、RTX 3090 上 54–190 倍），並指出正確的判準是 FLOPS 對主機鏈路之比而非 HBM 頻寬比。同一組量化分析亦顯示，既有 KV 卸載文獻的容量前提在大容量 HBM 加速器上不成立，問題性質由容量轉為機會成本，而在 24 GB 級加速器上則仍然成立（第 2 節）。
2. 我們系統性地比對了 15 篇近期工作，指出四項成本（未來效用、傳輸成本、重算成本、記憶體壓力）從未在單一線上決策中同時出現；現有最高覆蓋為 Marconi [35] 的三項，其因僅在 GPU 上運作而結構性地缺少傳輸成本（第 3 節）。
3. 我們將 KV 放置形式化為受品質約束的序列決策問題，動作空間包含六種狀態且明確納入「丟棄後重算」；並論證重算的價值來自其資源佔用特性而非單次成本，因而其正確性取決於未來效用預測（第 4 節）。
4. 我們提出 Tiara 的設計，透過 vLLM 的 CachePolicy 介面實作，可在 CUDA 與 ROCm 上零程式碼修改部署（ROCm 上的執行期限限制見第 7 節）。
5. 我們指出既有學習式 KV 管理方法所用的對稱損失函數，在動作成本相差 6–190 倍（隨平台而異）的設定下失準（10 倍不對稱下，Bayes 最佳門檻自 $p = 0.5$ 移至 $p \approx 0.09$ ），並提出將動作成本納入損失的兩種形式。經比對 8 個既有學習式系統，尚無工作將異質動作成本編碼進 KV 管理的損失函數（第 5.3 節）。
6. 我們以離線最優 oracle 量測此問題的改善空間，依預先訂定的判準給出條件式 GO，並給出兩個條件的可計算判別式：權重須量化（BF16 權重使 decode 佔端到端時間的 75%，而放置對 decode 無自由度），且請求長度須落在成本交叉點的 2–3.5 倍（第 6 節）。

本稿狀態。 本文為初稿。第 2 節的量化分析與第 3 節的文獻比對均已完成並經查證。第 6 節的每一個數字均可追溯至一條指令與一個輸出檔，凡未量到者標為 NOT_MEASURED；其中數項量測推翻了我們在動手之前的假設，論文按量測結果陳述，並在 §6.10 與第 7 節標明證據到哪裡為止。第 5 節的線上策略尚未實作，故本文不報告任何線上策略的效能；已設計但未執行的實驗列於附錄 B。我們在第 7 節說明此一狀態的意涵。

Table 1: 符號與代號。上半為全文使用的符號，下半為量測設定的代號。

符號	
$\mathcal{A}$	動作空間，式 (4) 的六個候選狀態
κ	重算對傳輸的成本比
P^*	Ssd 與 Drop 的成本交叉位置 (token)
α	重算成本對 block 絕對位置的斜率
ϵ	品質退化的上限，約束 $\Delta Q \leq \epsilon$
p_i	block i 於決策視窗內被再次存取的機率
$\hat{p}_i$	上式的校準後估計值
p^*	最佳決策門檻 $1/(1 + \kappa)$ ，式 (9)
ℓ_i, π_i	block i 的所在層級與精度
λ_m, λ_x	容量佔用與搬移成本的權重，式 (7)
模型剖面（權重精度 + 成本常數來源）	
qwen-awq	Qwen2.5-7B-Instruct-1M，AWQ-INT4（主設定）
llama-awq	Llama-3.1-8B-Instruct，AWQ-INT4 權重
llama-bf16	Llama-3.1-8B-Instruct，BF16 權重
Baseline	
full_gpu	不卸載，vLLM 預設
cpu_lru, cpu_arc	vLLM 內建的兩個 CPU 階策略
tier_fs	CPU 加磁碟兩階
lmcache	LMCache [2]
KV 精度（vLLM --kv-cache-dtype 的取值）	
auto	BF16，無損基準
fp8	純格式轉換，靜態縮放
int8_per_tok_head	per-token-head 動態縮放
int4_per_tok_head	同上，最低精度階

2 Background and Motivation

2.1 KV cache 的記憶體佔用

對採用 grouped-query attention (GQA) 的模型，每 token 的 KV 佔用為

$$b_{\text{tok}} = 2 \times L \times H_{kv} \times d_h \times s \quad (1)$$

其中 L 為層數， H_{kv} 為 KV head 數， d_h 為 head dimension， s 為每元素位元組數，前導的 2 對應 K 與 V。

表 2 給出代表模型的實際數值。¹

權重精度是正交的部署參數。 本文將 LLM 權重視為 frozen，且不修改其精度。式 (4) 的動作空間描述的是 **KV cache 的儲存精度**；權重精度（BF16 / FP8 / INT8 / AWQ-INT4）是另一個維度，不是本文的決策變數。我們因此不假設任何特定權重精度為「主流」。實驗在代表性的部署配置下進行——平台 A 採 AWQ-INT4（記憶體受限的消費級 GPU 之常見配置），平台 B 採 BF16 與 FP8——並將權重精度的影響列為敏感度分析（第 6 節），而非主要貢獻。將兩個維度混為一談會使本文同時變成權重量化、KV 卸載、驅逐與重算四個研究問題，模糊真正的貢獻。

¹組態取自 HuggingFace config.json（2026-08-29 查證）。六個模型的 torch_dtype 均為 bfloat16；BF16 與 FP16 同為 2 位元組，式 (1) 的算術不變，本文一律寫 BF16 以與發布的權重一致。

Table 2: KV footprint。^M 為 MoE。

模型	L	H_{kv}	d_h	KV/tok
Qwen2.5-7B-Inst-1M	28	4	128	56 Ki
Llama-3.1-8B-Inst	32	8	128	128 Ki
Qwen3-8B	36	8	128	144 Ki
Qwen3-14B	40	8	128	160 Ki
Qwen3-30B-A3B ^M	48	4	128	96 Ki
Llama-4-Scout ^M	48	8	128	192 Ki

Table 3: 單請求 KV footprint 與容納情形 (BF16 權重 + BF16 KV)。✓ 表示權重加 KV 可置入。

模型	ctx	KV	H100 80 GB	MI300X 192 GB
Llama-3.1-8B	128K	15.6 GB	✓	✓
	256K	31.2 GB	✓	✓
	1M	122 GB	×	✓
Qwen2.5-7B-1M	256K	13.7 GB	✓	✓
	1M	53.4 GB	✓	✓
Llama-3.1-70B	128K	39.1 GB	×	×

2.2 觀察一：單請求容量壓力在 MI300X 上幾乎不存在（但在 24 GB 級加速器上成立）

表 3 比較同一組模型在 H100 (80 GB) 與 MI300X (192 GB) 上的容納能力。此處固定為 **BF16 權重** 以隔離變因（權重精度的影響見上文與敏感度分析），並保留 10% 記憶體作為 activation 與碎片化餘裕。平台 A 的實際實驗採 AWQ-INT4 權重，其容納能力見第 2.6 節。

此表給出三個結論。

(a) 在 MI300X 上，8B–14B 模型於 128K 上下文僅消耗 HBM 的 8–10%。既有文獻普遍採用的「單請求、上下文遞增」實驗設定，在此硬體上量測不到記憶體壓力。相對地，既有的卸載瓶頸量測 [33] 顯示在 80 GB 級硬體上 99% 的延遲花在傳輸，但該結論建立在容量壓力存在的前提下。

(b) Llama-3.1-8B 的 1M token 上下文（122 GB KV 加 16 GB 權重）可完整置入單張 MI300X，而 H100 無法。若啟用 MI300X 原生支援的 FP8，Llama-3.1-70B 的 256K 上下文亦可置入。

(c) 單請求壓力在 MI300X 上並未消失，只是需要更大的模型或更長的上下文才能觸及；Llama-3.1-8B 需 1M 上下文（122 GB KV），Llama-3.1-70B 於 256K 需 FP8 才置入。**這正是本文在該平台的實驗設定**——同一條壓力軸推到這張卡的盡頭，而非改量並行度。並行度與跨 session 保留同樣會耗盡 HBM（8B 模型於 BF16 下約 10 個並行 128K 請求），但那是多租戶下的機會成本問題，與單請求的放置決策正交，本文不涵蓋（第 7 節）。

2.3 觀察二：卸載的相對代價隨硬體世代惡化

表 4 給出關鍵比值。

比值持續惡化的原因是結構性的：HBM 容量與頻寬逐代提升，而 PCIe Gen5 ×16 自 MI300X 至 MI355X 完全未

Table 4: 記憶體階層比值與計算-傳輸比。規格取自 [3, 9]。主機鏈路為單向理論值；FLOP/byte 為 dense BF16 峰值除以單向鏈路頻寬，代表一位元組穿越鏈路的時間內可 retire 的算術運算量。

	Mem BW	Host link	Mem:link	FLOP/byte
RTX 3090 ^{†‡}	0.94 TB/s	31.5 GB/s	30:1	2,254
H100 SXM	3.35 TB/s	64.0 GB/s	52:1	15,459
H200 SXM	4.8 TB/s	64.0 GB/s	75:1	15,459
MI300X	5.3 TB/s	63.0 GB/s	84:1	20,752
MI355X	8.0 TB/s	63.0 GB/s	127:1	—
GH200	~4 TB/s	450 GB/s	~9:1	—

[†] RTX 3090 使用 GDDR6X 而非 HBM，PCIe 4.0 ×16。[‡] RTX 3090 的 71.2 TFLOPS 為 dense BF16 峰值，其推導與一個常見的取值錯誤見附錄 A.4。

Table 5: 重算對傳輸的成本比 κ 。 κ 跨平台變動達 32 倍，這是本文「硬體條件化策略」主張最直接的量化依據。

平台	模型	重算	傳輸	κ
MI300X	Llama-3.1-8B	12.2 μ s	2.08 μ s	6×
	Llama-3.1-70B	107 μ s	5.20 μ s	21×
RTX 3090	Llama-3.1-8B	225 μ s	4.16 μ s	54×
	Llama-3.1-70B	1,972 μ s	10.4 μ s	190×

變。其後果是，任何卸載策略都必須掩蓋一個比 HBM 存取高近兩個數量級的延遲。

GH200 的 9:1 是例外：藉由 NVLink-C2C 的 450 GB/s，其卸載經濟性優於任何 x86 主機體的加速器約 7–14 倍。同一套 KV 放置策略不可能同時在 9:1 與 127:1 的平台上最佳，這是硬體條件化策略的直接論據。

2.4 觀察三：重算的價值不在單次成本

決定「重算或傳輸」的正確指標並非 HBM 頻寬比，而是 FLOPS 對主機鏈路之比，即一個位元組穿越 PCIe 的時間內，加速器能 retire 多少算術運算。MI300X 的 dense BF16 峰值為 1307.4 TFLOPS，故每傳輸一位元組可換得約 20,750 FLOP；H100 為 989.4 TFLOPS，對應 15,460 FLOP。MI300X 在此指標上優於 H100 約 1.34 倍。

然而，在絕對值上傳輸仍然較便宜，且便宜的程度隨平台劇烈變動。以簡化的 prefill 模型（線性層約 $2N_{\text{params}}$ FLOP/token，100% MFU）估算，成本比 $\kappa = t_{\text{recompute}}/t_{\text{transfer}}$ 如表 5。

且重算成本隨 block 的絕對位置增長，而傳輸成本則保持恆定。表 5 的數值應視為 κ 的下界。

此處須區分兩個容易混淆的量。整段 prefill 的總成本確為序列長度的二次式（attention 為 $O(L^2)$ ）；但本文的動作空間作用於固定大小的 block，而在位置 P 重算一個大小為 C 的 block，其 attention 需讀取前序的 P 個 token，成本為 $O(C \cdot P)$ ——對 P 為線性。我們在平台 A 上以 $C = 2048$ 、 $P \in \{0, 4K, 8K, 16K, 24K\}$ 實測 Llama-3.1-8B（每點三次取中位數），得

$$C_{\text{recompute}}(P) = 513.0 \text{ ms} + 26.9 \text{ ms} \times \frac{P}{1000}, \quad (2)$$

此式擬合自 llama-bf16 的五個位置 ($P \leq 24,576$)，最

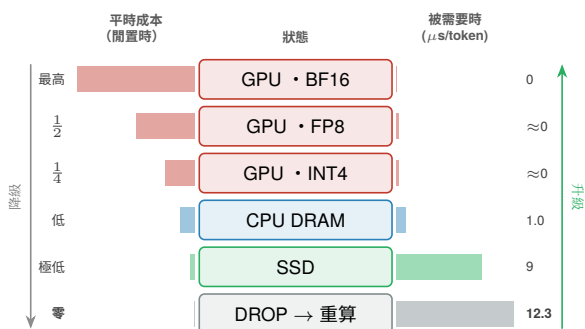


Figure 1: Tiara 的動作空間與其成本結構。與既有工作的差異有二：(i) 位置分層 (GPU/CPU/SSD，無損) 與精度分層 (BF16/FP8/INT4，有損) 被統一為單一階梯，而既有工作只做其中之一；(ii) Drop 是階梯上的一層，而非檢索失敗後的補救。兩側長條的長度即為成本，其反向增長是本文的出發點：Drop 閒置時不佔用任何資源，被需要時卻最貴，故其正確性完全取決於未來效用預測。此六階為典型配置之示意，量化器可替換 (見第 4 節)。

大偏差 1.6%，五點皆落於直線上，未觀察到二次成分；自 $P = 0$ 至 $P = 24,576$ 成本成長 2.29 倍。qwen-awq 另有十一個位置的量測 ($P \leq 258,048$)，線性形式在放大 10.5× 的位置範圍上仍成立，成本自 453.9 成長至 6,320.2 ms (13.9×)。係數依平台與模型而定 (表 12)，但線性這個形式來自 $O(C \cdot P)$ ，與平台無關。將重算視為單一常數會在長上下文下系統性低估其成本。

κ 亦隨模型架構變動，且方向可事先計算。 κ 的分子由啟用參數量決定，分母由 KV 大小決定，而 MoE 使兩者解耦：其 KV 由 attention 層決定 (與同規模 dense 模型相同)，但重算僅需啟用的專家。以 Qwen3-30B-A3B 為例 (48 層、4 個 KV head、128 個專家中啟用 8 個，啟用參數約 3B)，其 κ 在 RTX 3090 上為 27，僅為同平台 dense 模型 (48-108) 的約一半。

這產生一個可證偽的預測：在固定平台上，MoE 模型的最佳 Drop 觸發門檻應高於 dense 模型。第 6 節將檢驗之。此軸與硬體軸正交，兩者共同構成本文「成本結構決定策略」主張的雙重檢驗。

圖 1 揭示了一個關鍵的非對稱性：重算的價值不在於其單次事件成本，而在於它在被需要之前不佔用任何資源。因此重算的正確條件是「該 KV block 未來被存取的機率足夠低」——低到節省容量與頻寬能夠抵償萬一被需要時 6-190 倍的代價。

這個推論的重要後果是：未來效用預測不是系統的附加元件，而是決定重算動作正確與否的唯一依據。這與第 3 節的文獻分析指向同一結論。

2.5 觀察四：prefill 與 decode 的卸載經濟性相差五個數量級

同一筆搬移在 prefill 攤提於整段前綴，在 decode 則每產生一個 token 就重付一次。這個不對稱決定了動作空間的哪些層在何種工作負載下可用，而既有工作多半只在其中一側成立。

兩個階段的瓶頸不同。prefill 一次處理 N 個 token，算術强度高，受限於算力；decode 一次處理一個 token，但

每一步都須讀取整個 KV cache，算術強度趨近 1，受限於記憶體頻寬。我們在平台 A 上實測此差距 (Llama-3.1-8B 與 Qwen2.5-7B-1M，AWQ 權重，四個上下文長度)：

上下文	prefill (ms/tok)	decode (ms/tok)	倍數
16,384	0.316	17.0	54×
65,536	0.591	23.5	40×
126,976	0.903	29.7	33×
258,048	1.264	29.4	23×

此差距使卸載的成本結構在兩階段完全不同。以 Llama-3.1-8B 於 126,976 上下文、FP8-KV 計，KV 總量為 7.75 GiB。該平台的 PCIe 為 Gen3 ×16 (15.75 GB/s)，故將全部 KV 移至主機需 492 ms。在 prefill，此成本攤提於 126,976 個 token，等於每 token 0.0024 ms；在 decode，每一步都要重付，而全駐留於 HBM 的同一次讀取僅需 8.3 ms。

卸載比例	decode 每步增加	相對全駐留
5%	+24.6 ms	3.0×
20%	+98.4 ms	11.9×
100%	+492.1 ms	59.3×

在 full attention 下，decode 階段的卸載在任何比例都不可行。即使僅卸載 5%，每步仍付出全駐留 3 倍的代價。唯一可行的路徑是稀疏 attention——僅取回當步 attention 分數高的 block，即 Quest [43]、InfiniGen [25] 與 ArkVale [10] 的做法。

三種工作負載型態的有效動作空間不同。

型態	決策點	預測的對象	有效動作空間
prefill 為主	請求之間保留哪些前綴	該前綴是否再被請求 (時序)	六階全部
多輪對話	同上，另含思考空窗期間的保留成本	同上	六階全部
生成為主	請求之內哪些 block 可離開 GPU	該 block 的 attention 分數 (語意)	僅 GPU 內的精度階；CPU/SSD 慢 3-59×

Takeaway. 第 6 節的實驗涵蓋前兩種型態：其工作負載為「同一前綴送兩次」，prompt 與生成的比例為 1092:1，故 99.8% 的時間落在 prefill。第三種型態的決策點在請求之內，且預測的對象是 attention 分數而非存取機率——那是不同的預測問題，本文不處理。此界線與觀察三的 SSD 結果同構：六階是候選集合，其有效子集由平台與工作負載型態共同決定，而兩者皆可事先算出。

2.6 雙平台實驗設計

表 4 的 FLOP/byte 欄位揭示了一個跨越一個數量級的光譜：RTX 3090 為 2,254，MI300X 為 20,752，相差 9.2 倍。此差距的來源是結構性的：3090 的 GDDR6X 頻寬與 PCIe 4.0 鏈路較為接近，而 MI300X 的 HBM3 遠快於其 PCIe 5.0 鏈路。

這使得本文的核心主張成為可證偽的：若最佳 KV 放置策略確實隨硬體比值結構性改變，則同一組策略在相

距 9.2 倍的此二平台上應展現不同的最佳動作組合。須說明的是，此二平台位於該光譜的中段而非兩端（表 4 中 GH200 為 9:1、MI355X 為 127:1），我們的可證偽性主張因而受限於實際可取得的硬體跨度。具體而言，我們預測 Drop→重算動作在 MI300X 上的最佳觸發門檻，應顯著低於在 RTX 3090 上的門檻。

兩個平台承擔的是同一條壓力軸的兩端：RTX 3090 (24 GB GDDR6X) 可推至 8B 模型的約 262K 上下文，MI300X (192 GB HBM3) 則延伸至更大的模型與 1M 級上下文。這不是資源限制下的妥協：單一平台無法檢驗硬體條件化的主張，因為該主張的內容正是「常數改變時最佳策略隨之改變」。

平台 A 上 64K 與 128K 之間存在一道明確的容量懸崖，既有文獻所預設的場景在此平台上完全成立；實測的逐設定容量見 §6.9。

3 Related Work

我們依動作與訊號兩個維度組織既有工作，並在表 6 給出特徵矩陣。判準從嚴：「未來預測」欄僅在該工作預測尚未發生的存取時才標記，僅使用當前步 attention 分數者不計。

3.1 分層儲存

FlexGen [39] 首次以線性規劃決定 KV 在 GPU/CPU/disk 三層間的比例配置，但其策略為離線靜態。CachedAttention [17] 針對多輪對話保留 session 級 KV 於 HBM/DRAM/SSD，並利用排程器佇列進行預取。Mooncake [36] 將叢集中間置的 CPU/DRAM/SSD 池化為共享 KVCache 層。KVDrive [28] 以 prefill 階段的 attention 重要性剖析決定 HBM/DRAM/SSD 的初始配置，並將全量 KV 寫入 SSD 作為後備。Tutti [37] 則專注於 SSD I/O 路徑本身，透過 GPU 發起的儲存存取消除 CPU 瓶頸。

這些工作共同的限制是：放置訊號來自當前或過去的觀測，且不將重算視為動作。

3.2 可恢復驅逐

ArkVale [10] 觀察到 token 重要性隨解碼步驟變動，早期被驅逐者可能重新變得重要，因而以 page digest (key 向量的包絡體) 估計已驅逐頁的重要性並召回。QEvict [19] 則將可恢復性建立在精度維度：中等信心的視窗被量化保存，重新變重要時反量化升回全精度。

這兩類工作分居兩個正交維度：ArkVale 及其後繼者 [42, 43] 做位置分層 (GPU↔CPU，且兩層皆為全精度)，QEvict 做精度分層 (全精度↔量化，但僅一個位置)。據我們所知，尚無工作以單一重要性驅動策略統一管理兩者。

3.3 學習式未來效用預測

2026 年出現多篇以學習方法預測 KV 未來效用的工作：KVP [34] 以 per-head 的強化學習代理排序 token，其 reward 直接導自未來效用；ForesightKV [11] 建構以真實未來 attention 分數計算的 golden eviction oracle 並以模仿學

習逼近；LookaheadKV [4] 則以可學習的 lookahead token 預測模型真實回應所導出的重要性。Marconi [35] 的效用函數同時包含預測重用機率、計算節省與記憶體佔用。

這些工作的動作空間均為單層的保留/丟棄，且無一支援恢復。最強的未來預測器沒有安全網，而最完善的安全網沒有未來預測器。

3.4 重算與傳輸

Cake [23] 讓 GPU 正向計算與 I/O 反向載入同時進行並在中點會合。KVPR [21] 以 profiler 與 scheduler 求解活化值傳輸與 KV 重算的最佳切分。HCache [18] 儲存中間 hidden state (約為 KV 的一半大小) 並由其重建 KV，實質上在階梯上增加了第三階。

這些工作共同的時序特性是檢索時的反應式決策：它們回答「此刻需要這段 KV，如何最快取得」，而非「未來可能需要，現在應置於何處」。ACL 2026 的綜述 [22] 中，整理「GPU KV 重算 ↔ KV 傳輸」重疊的表格僅列出一個範例 (KVPR)。

以 query 為訊號者無法取代放置決策。Quest [43]、InfiniGen [25]、ArkVale [10] 與 ShadowKV [42] 皆以當下的 query 向量估計各 block 的重要性並取 top- k 。這是取得單步 attention 稀疏性最直接的訊號，其準確度上限也高於任何以存取歷史為輸入的預測器。然而該類方法在本文的問題上有兩個結構性障礙。

(i) query 只覆蓋當下這一步。 $q \cdot k$ 回答的是「此刻這個 query 需要哪些 KV」。跨請求的重用——一段前綴在數分鐘後被另一個尚未抵達的請求命中——所對應的 query 尚不存在，因而無法由此類訊號涵蓋。這正是上一段所述反應式與前瞻式的分界。

(ii) 計算 $q \cdot k$ 預設 k 仍可取得。要以 query 判斷是否保留某 block，必須先讀取該 block 的 key；而是否保留該 key 正是待決策的變數。此依賴使 query 訊號無法用於已降級至 SSD 或已丟棄的 block，亦即無法用於階梯下緣——而階梯下緣正是成本差異最大之處。以 query 為訊號的方法因此運作於「內容皆可取得，僅需挑選搬運對象」的前提之上，其前一步（內容該置於何處、是否保留）即為本文所處理的問題。

本文不將二者視為競爭關係：query 側訊號可作為特徵併入前瞻式預測器（第 5.2 節），對仍可取得的 block 提供高品質輸入，而預測器負責覆蓋 query 無法觸及的跨請求與已降級情形。

3.5 品質作為約束

KVServe [30] 將壓縮 profile 選擇形式化為 $\arg \min_p T_p(c)$ s.t. $T_p(c) \leq T_{SLO} \wedge q_p(w) \geq q_{min}$ ，是我們所知唯一在 Tier-1 場所將模型品質寫為硬性約束的 KV 工作，但其決策變數不含放置。EvicPress [16] 與 AdaptCache [15] 將品質納入效用函數，但作為可調權重而非約束。LeoAM [26] 是我們調查中唯一給出數值上界者（宣稱準確度下降小於 1%）。OrbitFlow [32] 以 ILP 於延遲 SLO 與記憶體容量約束下決定逐層放置，但因其為無損設計，ILP 中不存在品質變數。

3.6 形式化與上界

Fang 等人 [14] 將異質記憶體系統中的 KV 放置問題數學形式化並推導理論上界，明確指出「存在可觀的執行期最佳化空間」，但同樣明確表示不提出具體排程策略。本文正是填補其留下的策略空缺。

3.7 特徵矩陣

3.8 研究缺口

由表 6 可得三項觀察：

(1) 「重算」與「品質約束」兩欄從未在同一列同時標記。

(2) 四項成本從未同時進入單一線上決策。傳輸成本見於 CacheGen 一系，重算成本見於 Cake/KVPR/HCCache，記憶體成本見於 EvicPress，未來效用見於 Marconi/KVP。最高覆蓋為 Marconi 的三項（未來效用、計算節省、記憶體佔用），其因僅在 GPU 上運作而結構性地不可能具備傳輸成本。

(3) 兩條研究血脈互不交會。以 FlexGen 為基礎的一系（IMPRESS、LeoAM、InfiniGen）具備重要性感知與有損處理，但無跨請求重用；以 vLLM 為基礎的一系（CacheBlend [46]、LMCache [2]、Cake、MTDS）具備跨請求重用，但將 KV 視為精確且不可分割。尚無系統同時具備重要性感知的有損分層與跨請求重用。

綜述 [22] 自身的 Observation O7 指出壓縮與放置的協同設計是「a missed opportunity」，Challenge C5 呼籲在共享預算下聯合最佳化 eviction、offload 與 prefetch，而該清單未包含重算。

4 Problem Formulation

4.1 狀態與動作

設 KV cache 被劃分為 block 集合 $\mathcal{B}$ 。於決策時刻 t ，系統狀態為

$$s_t = \left(\{\ell_i, \pi_i, a_i, n_i\}_{i \in \mathcal{B}}, C^{\text{gpu}}, C^{\text{cpu}}, C^{\text{ssd}}, Q_{\text{pcie}}, \phi_t \right) \quad (3)$$

其中 ℓ_i 為 block i 的所在層級， π_i 為其精度， a_i 為上次存取時刻， n_i 為 token 數， C^* 為各層剩餘容量， Q_{pcie} 為互連佇列深度， ϕ_t 為當前查詢特徵。

動作空間為

$$\mathcal{A} = \{\text{Gpu}_{16}, \text{Gpu}_8, \text{Gpu}_4, \text{Cpu}, \text{Ssd}, \text{Drop}\} \quad (4)$$

對應圖 1。Drop 動作意味著該 block 不被儲存於任何位置，若後續被需要則由 token 重算。Cpu 與 Ssd 儲存未經壓縮的位元組——二者只改變 block 所在的位置，不改變其內容；此無損性於 §6.6 以逐字元比對驗證 (60/60)。精度維度因而只存在於 GPU 常駐的三階， ϵ 約束的實質內容亦全部落在該三階上。

階梯是示意，量化器可替換。式 (4) 的六個狀態代表典型部署選項，而非本文主張的特定量化方案。實務上 KV 量化存在一個重要的非對稱性：key 對量化誤差遠比 value 敏感，故生產系統普遍採 K/V 分離的精度配置——

LMCache 的預設 turboquant_k8v4 為 key 8 位元、value 4 位元；KIVI [31] 對 key 採 per-channel、對 value 採 per-token 量化，因二者離群值結構不同。

本文的決策框架與此正交：任何量化器只要能給出「容量佔用」與「存取延遲」兩個數字，即可作為階梯的一階。將 Gpu_8 替換為 K8V4、或將 Gpu_4 替換為 KIVI 的 2 位元方案，均不改變式 (8) 的形式，只改變成本常數。

Drop 的可行性受前序 block 約束。其餘五個動作彼此獨立——一個 block 放在哪一層，不影響其他 block 能否被取回。Drop 是唯一的例外。

重算 block i 並不需要自 token 0 重跑整個序列：以 chunked prefill 的方式，只需將 block i 的 token 前向通過模型，其 attention 讀取前序 block 既有的 KV。這使重算成本為 $O(|b_i|)$ 而非 $O(i)$ 。但這也意味著：

$$a_i = \text{Drop} \text{ 可行} \iff \forall j < i: \text{block } j \text{ 的 KV 可取得} \quad (5)$$

若前序 block 亦被丟棄，必須先重算它們，成本沿依賴鏈累積：

$$\begin{aligned} \text{recomp}(i) &= \sum_{j \in \mathcal{D}(i)} \left(2N_{\text{active}}|b_j| + \text{attn}(j) \right) \\ \mathcal{D}(i) &= \{i\} \cup \{j < i : a_j = \text{Drop}\} \end{aligned} \quad (6)$$

此約束的意涵是 Drop 決策彼此耦合，不可獨立最佳化。一個只看單一 block 效用的策略會系統性地低估連續丟棄的代價。我們以「同一序列內連續 Drop 的 block 數上限」作為可行性約束處理之，並於第 6 節量測該上限對 Pareto 前緣的影響。這也是 HCCache [18] 儲存 hidden state 的動機：由 hidden state 可單步重建 KV，從而切斷依賴鏈。

4.2 成本模型

對 block i 採取動作 a 的即時成本為

$$\begin{aligned} c(i, a) &= \underbrace{\lambda_m \cdot \text{mem}(i, a)}_{\text{容量佔用}} + \underbrace{\lambda_x \cdot \text{xfer}(i, a)}_{\text{搬移成本}} \\ &\quad + \underbrace{p_i \cdot [\text{fetch}(i, a) + \text{recomp}(i, a)]}_{\text{期望存取成本}} \end{aligned} \quad (7)$$

其中 $p_i = \Pr[\text{block } i \text{ 於未來視窗內被存取}]$ 為未來效用預測，fetch 為自 ℓ_i 載回 GPU 的延遲，recomp 僅在 $a = \text{Drop}$ 時非零，且依第 2.4 節之模型隨 block 的絕對位置增長。

4.3 受約束的最佳化問題

給定品質預算 ϵ ，

$$\begin{aligned} \min_{\{a_i\}} \quad & \sum_{i \in \mathcal{B}} c(i, a_i) \\ \text{s.t.} \quad & \Delta Q(\{a_i\}) \leq \epsilon \\ & \sum_i \text{mem}^{\text{gpu}}(i, a_i) \leq C^{\text{gpu}} \quad (\text{對 CPU/SSD 同理}) \\ & \text{TTFT} \leq T_{\text{SLO}}, \text{TPOT} \leq P_{\text{SLO}} \end{aligned} \quad (8)$$

式 (5) 的可行性約束

Table 6: 特徵矩陣。● 表示明確支援，○ 表示部分或間接支援。判準從嚴（見正文）。「未來預測」不含僅使用當前步 attention 估計者。

工作	場所	GPU/CPU	SSD	未來預測	壓縮	恢復	重算	品質約束
FlexGen [39]	ICML'23	●	●		●	●		
InfiniGen [25]	OSDI'24	●		○		●		
ArkVale [10]	NeurIPS'24	●				●		
Cake [23]	ICML'25	●	●			●	●	
HCACHE [18]	EuroSys'25	●	●		○	●	●	
Marconi [35]	MLSys'25			●			○	
ShadowKV [42]	ICML'25	●			●	●	○	
OrbitFlow [32]	VLDB'26	●				●		
KVserve [30]	SIGCOMM'26	●			●			●
KVP [34]	ICML'26			●				
Het-Mem [14]	IEEE CAL'25	●				●		
MTDS [44]	C&IS'26	●	●	○		●	●	
KVDrive [28]	preprint	●	●			●		
EvicPress [16]	preprint	●	●	○	●	●		○
QEvict [19]	preprint				●	●		
Tiara (本文)	—	●	●	●	●	●	●	●

此形式化與既有工作的差異有三：(i) 動作空間 (4) 同時跨越位置與精度兩個維度，並將 Drop 視為一個具明確標價的層級而非失敗路徑；(ii) 品質為硬性約束而非目標函數中的加權項，此點與 KVserve [30] 一致但作用於放置而非壓縮；(iii) 成本函數 (7) 中四項成本同時出現。

€ 的可操作性。 ΔQ 於執行期不可直接觀測，這是所有品質約束工作面臨的共同困難。KVserve 採用離線 profiling 加上 ϵ -greedy bandit 修正。我們規劃在第 6 節評估兩種代理訊號：(a) 離線 profile 建立的 (壓縮率, 任務類型) $\rightarrow$ 品質查找表；(b) 模型輸出分佈的信心值作為線上回饋。此為本設計最主要的風險點，我們於第 7 節說明。

5 Design

5.1 系統架構

圖 2 給出整體架構。設計上的關鍵決定是：不自行實作推論引擎。vLLM [24] 的 OffloadingConnector 已提供 GPU $\rightarrow$ CPU $\rightarrow$ (FS/物件儲存/P2P) 的搬移機制，且官方文件明確載明支援 CUDA、ROCm 與 XPU；其上游 CI 於實體 gfx942 硬體上執行 KV offload 測試。該連接器內建的置換策略僅有 lru 與 arc，並透過 CachePolicy 介面開放自訂實作。

此決定有三個後果：(i) Tiara 可在 CUDA 與 ROCm 兩個平台上零移植部署；(ii) 對照組為 production 級實作而非我們自行撰寫的弱化版本；(iii) 貢獻邊界清晰：機制屬於 vLLM，策略屬於本文。

5.2 未來效用預測器

預測器輸出 p_i ，即 block i 於未來視窗內被後續請求命中的機率。此定義的範圍須明確： p_i 是跨請求的時序量，而非請求之內的 attention 分數。在 full attention 的 decode 階段，每個 block 於每一步皆被讀取， p_i 恆為 1，預測無

Table 7: 既有學習式快取／放置預測器的設計比對。最後一欄為本文關注的重點：損失函數是否對不同錯誤方向給予不同代價。

系統	模型	動作數	成本不對稱
KVP [34]	112 $\times$ MLP	2	否
ForesightKV [11]	MLP(16)	2	部分 (單邊)
LookaheadKV [4]	LoRA $r=8$	2	否
Marconi [35]	啟發式	2	否 (但有 value/byte)
Mockingjay [38]	表格 + TD	2	否
LRB [41]	GBDT	2	否 (L2)
PARROT [29]	LSTM+attn	2	是 (NDCG gain)
Sibyl [40]	C51 DQN	n	是 (實測延遲)
Tiara (本文)	GBDT	6	是 (動作成本加權)

從施力 (第 2.5 節)。請求之內的取捨由稀疏 attention 處理，其預測對象為 $q \cdot k$ ，屬不同問題。我們對 8 個既有的學習式快取／放置系統做了設計層級的比對 (表 7)，其結果約束了預測器的四個面：預測目標、特徵集、模型族，以及訓練資料的取得方式。

規格。 預測器是一個獨立於 LLM 之外的小型模型，不參與 attention 計算，亦不修改模型權重。其介面如下：

模型為單一全域 GBDT， $layer_id$ 與 $head_id$ 作為特徵而非模型的依據。

門檻永遠施加於 $\hat{p}_i$ ，不施加於 $\hat{y}_i$ ——式 (7) 與式 (9) 使用的皆為機率。此區分是必要的：對 $\hat{y}_i$ 直接閾值化在數學上不對應式 (9) 的最佳邊界 (見第 5.3 節)。

精度只能往下，位置可以來回。圖 3 給出動作 a_i 的三個面。其一，作用範圍是 16 token 的整欄 (圖 3a)：vLLM 的 block table 以 token 為索引且踰層共用，故 $layer_id$ 與 $head_id$ 只能作為特徵，不切分動作空間。其二，任一時刻的垂直切片上不同 block 位於不同狀態 (圖 3b)，

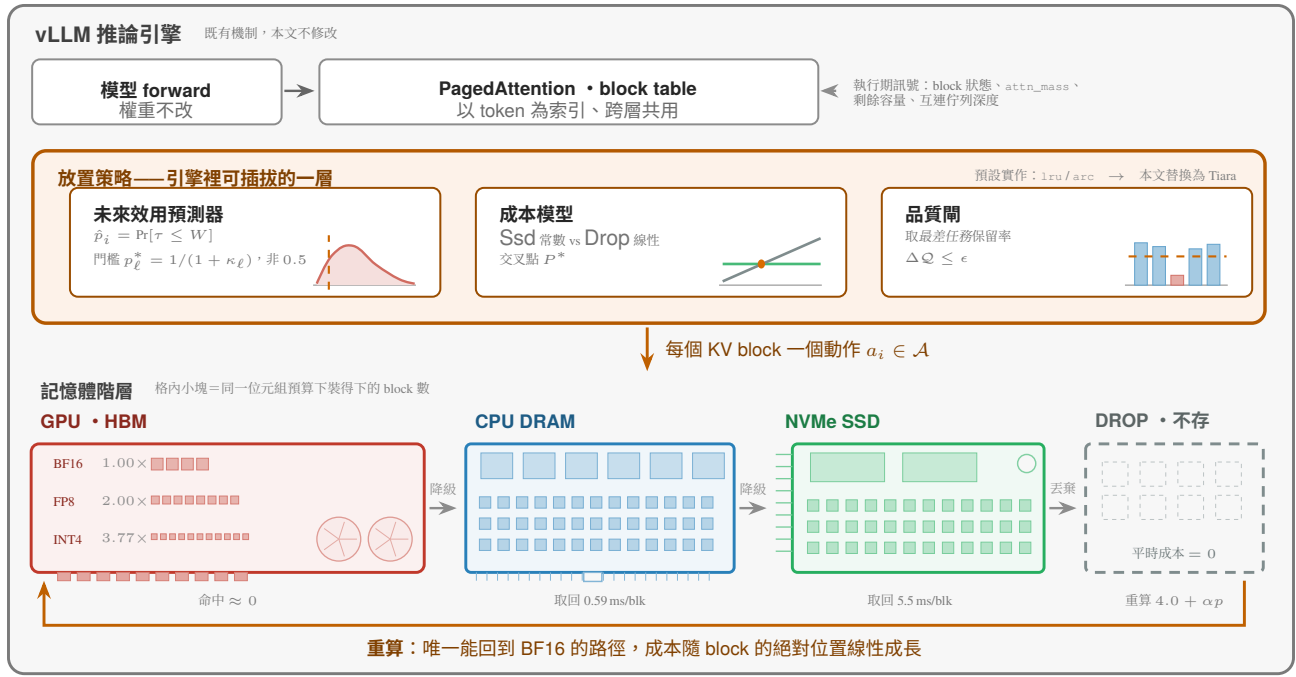


Figure 2: Tiara 在 vLLM 裡的位置。整張圖都在 vLLM 引擎之內：模型 forward、PagedAttention 的 block table、以及底層的記憶體階層皆為既有機制，本文不修改。本文替換的是中間那一層——引擎原本就開放的可插拔放置策略，其預設實作只有 lru 與 arc。該層接收四類執行期訊號，經「未來效用預測 → 成本模型 → 品質閘」為每個 KV block 產生一個動作 $a_i \in \mathcal{A}$ 。三個階段各以其判準的形狀表示：預測器的門檻取自式 (9) 的 $p_i^* = 1/(1 + \kappa_\ell)$ 而非 0.5；成本模型中 Ssd 為常數而 Drop 隨位置線性成長，二者交於 P^* ；品質閘取最差任務的保留率（式 (14)）。底層四個狀態即動作空間 (4)：精度的三階同住在 GPU 上（格數比為實測容量倍數，§6.9），Drop 則平時成本為零而取回最貴。機制屬於 vLLM，策略屬於本文，故 Tiara 是一個 plugin 而非 fork。

故 a_i 是逐 block 的決策而非全域設定。其三，位置與精度兩個維度的可逆性不同（圖 3c）：位置搬移只是位元組往返（§6.6 以 60/60 逐字元比對驗證），因而可逆；而精度降級丟棄位元，階梯上沒有任何一條「精度往上」的轉移——唯一能回到 Gpu₁₆ 的路徑是 Drop 之後重算，其成本隨 block 的絕對位置線性成長（式 (2)）。此不對稱是預測器設計的一個前提：位置維度上的一次誤判可以廉價更正，精度維度上則不行。

標籤的取得比決策慢一整個視窗。 決策是 t_d 當下的一次前向流程（演算法 1），約 $30 \mu s$ 且執行於 CPU；但標籤 τ_i 的定義涉及未來，無法於同一時刻取得——樣本須進入待標記佇列，由「下次被存取」或「等滿視窗 W 仍未被存取」兩個機制之一完成標記（見本節稍後〈標籤〉一段）。兩者的時間尺度不同，混為一談會誤以為訓練集可由「後來確實被重用的 block」單獨構成。

預測目標是連續量，而非二元分類。 所有預測連續量並延後閾值化的系統，都勝過其二元分類的前身，且 Mockingjay [38] 與 LRB [41] 均明確報告此點。LRB 嘗試過對 Belady 邊界的二元分類並依 good-decision-ratio 拒絕之。我們因此回歸 log(距下次存取的時間)。取對數的理由與 LRB 相同：真正重要的是落在邊界的哪一側，100K 與 2M 的差異遠比 2M 與 3M 重要。

三族存取歷史特徵，服務時取得成本為零。 三族皆沿用 LRB [41] 的特徵設計：(i) *deltas*，即距該 block 前 k 次

存取的時間間隔 ($k \leq 32$ ，未填滿者為 ∞)，此設計涵蓋 LRU、LRU- K 與 S4LRU；(ii) *EDC*，即 10 個指數衰減計數器 $C_i \leftarrow 1 + C_i \cdot 2^{-\Delta_i/2^{9+i}}$ ，僅在存取時更新；(iii) 靜態特徵，即 block 大小、layer id、head id 與 token 類型。最後一項的依據是 SAECache [13] 量到不同語意類型 (system prompt / 使用者查詢 / 工具輸出 / 推理鏈) 的重用率差異達 756 倍，而該特徵取得成本為零。

不看 KV 而預測 KV 的重用站不住腳。 上述特徵全部來自存取歷史，不含模型自身的表徵。這與三個學習式先例形成對比：KVP [34] 明確「僅使用 key 與 value 向量」，LookaheadKV [4] 透過層內 adapter 讀取，ForesightKV [11] 蒸餾真實的未來 attention。「在不看 KV 的情況下預測 KV 的重用」作為服務層的低成本選擇是合理的，但作為學習貢獻則站不住腳。我們因此將 block 內 key/value 向量的池化統計併入特徵集，並在第 6 節以消融比較「僅存取歷史」與「存取歷史 + 模型內部表徵」兩種特徵集。

query 側訊號對仍位於 GPU 的 block 免費且更準。 第 3 節論證以 query 為訊號者無法取代放置決策，原因是 $q \cdot k$ 對已降級的 block 不可得。但對仍位於 GPU 的 block，該訊號的品質高於任何存取歷史特徵，捨棄它並無理由。我們因此記錄 *attn_mask*，即最近 8 個解碼步中該 block 取得的 attention 質量佔總質量之比。此量在 forward pass 中已被計算，記錄它僅需一次 per-block 的規約，不引入額外的 $q \cdot k$ 。

此設計使特徵集在階梯上呈降級退化的形態：位於 GPU 的 block 同時具備存取歷史與 query 側訊號，位於

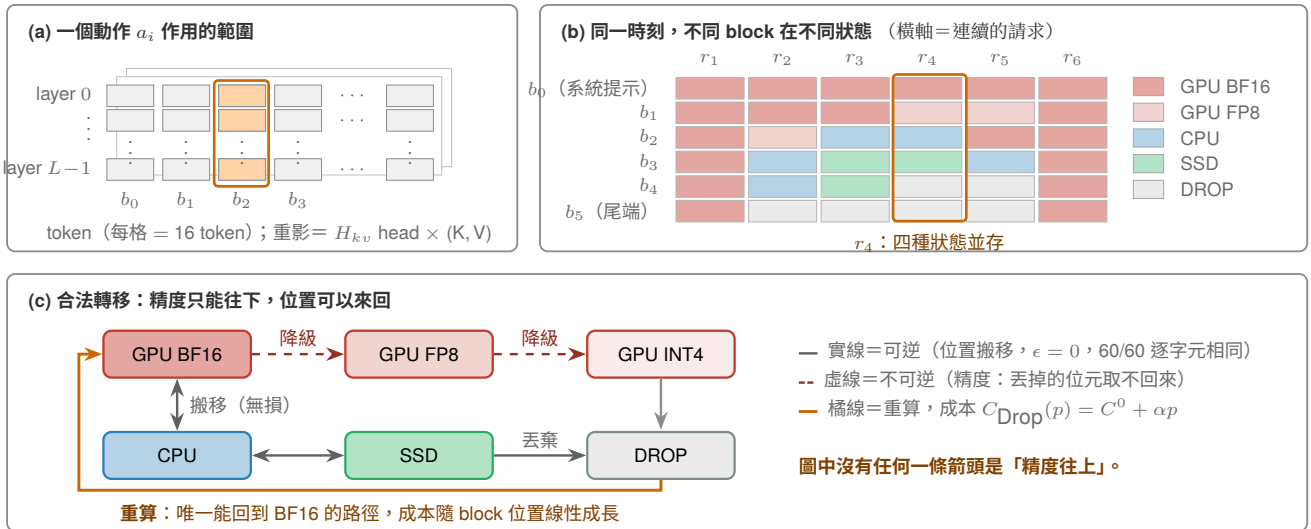


Figure 3: 動作 a_i 是什麼。(a) 一個動作作用的範圍：vLLM 的 block table 以 token 為索引且跨層共用，故一個 a_i 同時套用於 16 token 的全部 layer、全部 KV head 與 K/V 兩者（橘色整欄）；layer_id 與 head_id 僅作為預測器的特徵 (§5.2)，不切分動作空間。(b) 同一前綴的六個 block 在六個連續請求中所處的狀態：任一時刻的垂直切片上不同 block 位於不同狀態，這正是 a_i 為逐 block 決策而非全域設定的原因（示意，非量測資料）。(c) 狀態之間的合法轉移：位置的三階之間只是位元組往返，§6.6 以 60/60 逐字元比對驗證為無損，故可逆；精度降級丟棄位元，故單向。圖中沒有「精度往上」的箭頭——唯一能回到 GPU₁₆ 的路徑是 Drop 之後重算，其成本隨 block 的絕對位置線性成長（式 (2)）。此不對稱有兩個後果： ϵ 約束只作用於虛線，故 §6.2 的 oracle 在四個無損動作上求解時 ϵ 自動滿足；以及預測器的一次誤判在位置維度上可廉價更正、在精度維度上則不行。

Algorithm 1 預測器的服務時流程（每個 KV block 一次）

Require: block b 、決策視窗 W 、GBDT f 、校準映射 g 、各階成本比 κ_ℓ

Ensure: 動作 $a \in \mathcal{A}$

特徵抽取

▷ 六族特徵，全部於服務時免費取得

```

1: deltas  $\leftarrow$  前 32 次存取的時間間隔          ▷ 未填滿者為  $\infty$ 
2: edc[i]  $\leftarrow 1 + \text{edc}[i] \cdot 2^{-\Delta_1/2^{9+i}}$ ,  $i = 0..9$   ▷ 僅於存取時更新
3: static  $\leftarrow$  (block_size, layer_id, head_id, token_type)
4: pooled  $\leftarrow$  block 內 key/value 向量的池化統計
5: attn  $\leftarrow$  最近 8 個解碼步中  $b$  取得的 attention 質量佔比  ▷ forward 已算出
6:  $\phi \leftarrow \text{concat}(\text{deltas}, \text{edc}, \text{static}, \text{pooled}, \text{attn})$ 
   評分                                ▷ 迴歸連續量，不做二元分類
7:  $\hat{y} \leftarrow f(\phi) = \log \hat{\tau}$               ▷  $\hat{\tau}$  為距下次存取的時間
8:  $\hat{p} \leftarrow g(\hat{y}) = \Pr[\tau \leq W]$   ▷ isotonic regression, 驗證集上校準
   依成本設門檻                        ▷ 門檻由式 (9) 給出，非 0.5
9: for 每個階  $\ell$ ，由廉至貴 do
10:   if  $\hat{p} \geq p_\ell^* = 1/(1 + \kappa_\ell)$  then
11:     return  $a \leftarrow \ell$ 
12:   end if
13: end for
14: return  $a \leftarrow \text{Drop}$ 

```

CPU/SSD 者僅有存取歷史，已丟棄者僅有靜態特徵。GBDT 原生處理缺失值，此結構無須額外處理。我們在第 6 節以消融量測 attn_mass 的邊際貢獻。

選 GBDT 而非神經網路，因預測器的成本自其收益中扣除。 LRB 明確比較過 GBM、線性迴歸、邏輯迴歸、SVM 與 2 層 16 隱藏單元的 NN，GBM 在不同 trace 與快取大小下穩定勝出，且訓練僅需 300ms、對 64 個候選推論僅需 30 μs 。GBDT 亦能原生處理 ∞ 值的 delta 特

徵。我們採用單一全域模型並以 layer/head id 作為特徵；KVP [34] 的 112 個獨立 agent 帶來可觀的工程成本，而其論文並未提供「per-head 模型優於帶 head-id 特徵的共享模型」的消融證據。

此決定另有兩個本設定特有的理由。(i) 預測器的成本自其收益中扣除。預測器運行於服務熱路徑，其延遲直接抵銷所避免的重算或傳輸。以表 5 的區間為參照，單次重算為 12–1,972 μs ，而對 64 個候選的 GBDT 推論為 30 μs ；若改以需要一次 forward pass 的模型評分，僅 kernel launch 與排隊即進入相同數量級，收益隨即被抵銷。(ii) 神經預測器與主模型競爭同一加速器。GBDT 於 CPU 上執行，完全不佔用 GPU；任何在 GPU 上評分的預測器都會插入主模型的 decode 批次，其代價不僅是預測器本身的延遲，還包括對批次結構的擾動。

此外，主模型規模上升時此權衡朝有利於 GBDT 的方向移動： κ 隨模型增大而上升（表 5：8B 至 70B 由 6 增至 21，於平台 A 由 54 增至 190），而 GBDT 的推論成本不隨模型規模改變，因其特徵取自存取歷史而非模型參數。

我們不要求讀者接受此論證，而以實驗回答：表 15 的 (D) 區塊於相同特徵集與相同損失下比較四種預測器，並同時量測品質收益與熱路徑延遲。

前三者（GBDT、小型 MLP、其容量放大一階者）皆接受攤平後的特徵向量，故僅檢驗容量。第四者為直接消費 deltas 時間序列的小型 GRU，檢驗的是結構——即時序資訊應由手工特徵編碼（本文與 LRB 的作法），抑或應由模型自行學習。此對照不可省略：LRB 所比較的 2 層 16 隱藏單元 NN 同樣接受攤平輸入，其「GBM 勝出」的結論因而不涵蓋序列模型，本文若僅援引該結論即屬過度推論。

deltas 與 EDC 事實上已是時序的手工編碼——前者為存取間隔序列的截斷，後者為 512 至 262,144 十種半衰

期上的指數加權和，其功能與循環網路的隱藏狀態同構，差別在於前者的更新規則寫死而後者由訓練得到。(D) 區塊所檢驗的正是此差別是否重要。若 GRU 在計入自身延遲後仍佔優，第 5.2 節的整條特徵工程路線即被推翻，我們將據以修改。

標籤：負樣本只能由滑動視窗取得。 標籤 τ_i 無法在記錄特徵的當下取得，因其定義涉及未來。我們於取樣時記錄特徵並將該樣本置入待標記佇列，其後由兩個機制之一完成標記：(a) 該 block 下次被存取時，以兩次存取的間隔為標籤；或 (b) 當該樣本離開一個長於 relaxed-Belady 邊界的滑動視窗 W 而仍未被存取時，立即以「 $> W$ 」標記。

機制 (b) 是取得負樣本的唯一途徑。僅有機制 (a) 時，訓練集只會包含「後來確實被重用」的 block——從未被重用者永遠等不到第二次存取，因而永遠不進入訓練集。模型將完全未見過應被丟棄的樣本長何模樣，而那正是 Drop 動作的目標群體。取樣以 block 為單位而非以請求為單位，以避免偏向熱門 block。

隨機切分會洩漏未來，故依時間切分。 訓練與測試集依 trace 的時間順序切分（前 70% 訓練、後 30% 測試），不作隨機切分。隨機切分會使訓練集含有測試期之後的存取事件，而 EDC 與 delta 特徵本身即為時間的函數，此洩漏將系統性地高估預測品質。此點在快取預測文獻中屬常識，但其違反不易由結果察覺，我們因此明確記載。

飄移由 token 類型特徵與週期性重訓吸收。 不同工作負載的重用結構不同：多輪對話的前綴重複命中，RAG 的文件塊多為一次性讀取，agent 流量則呈現極熱的系統提示與極冷的工具輸出。SAECache [13] 所量到的 756 倍重用率差異即為此分布差異的直接證據。我們以三個層次因應：(i) token_type 作為特徵，使模型得以在單一模型內區分語意類型；(ii) 週期性重訓，利用 GBDT 300 ms 的訓練成本，以最近一段時窗的樣本重新擬合；(iii) 第 6 節報告跨工作負載泛化，即於工作負載 A 訓練並於 B 測試的退化幅度。

第 (ii) 項是 GBDT 選擇的直接後果而非附帶效果。KVP [34] 的 112 個強化學習 agent 需 8 GPU DDP 訓練，其重訓週期以日計，結構上無法追隨以分鐘計的流量變化。對分布飄移的適應成本，是預測器族選擇的一部分，而非可事後補救的工程細節。

5.3 成本敏感的損失函數

這是本文與既有工作差異最大的一點，也是我們認為最具貢獻的設計。

問題：對稱損失在本設定下可證明地失準。 既有 KV cache 的學習式方法 (KVP、LookaheadKV、SP-KV) 與經典快取方法 (LRB、Mockingjay) 的損失函數對兩種錯誤方向給予相同代價。在單一動作（驅逐）的設定下這是合理的。但本文的動作空間有六個成本迥異的選項，兩種錯誤的下游代價相差 6–190 倍，且該比值本身隨平台變動達 32 倍（表 5）：

- **False negative**（預測不會被使用 → Drop → 後續被需要）：被迫重算，12–108 $\mu\text{s}/\text{token}$
- **False positive**（預測會被使用 → 保留 → 實際未使用）：僅浪費容量

令 $\kappa = c_{\text{FN}}/c_{\text{FP}}$ 為成本比。保留的期望成本為 $(1 - p)c_{\text{FP}}$ ，丟棄的期望成本為 pc_{FN} ，兩者相等處即為最佳決策門檻：

$$p^* = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = \frac{1}{1 + \kappa} \quad (9)$$

這是成本敏感學習的標準結果 [12]，我們不主張其為本文貢獻。其意涵是：在 $\kappa = 10$ 時 $p^* = 1/11 \approx 0.09$ 。一個以對稱損失訓練、並在 0.5 處閾值化的預測器，其決策門檻被錯置了約一個數量級。

本文的主張是更具體的一點：既有所有學習式 KV 管理方法的動作空間都是二元且成本同質的，因此 $\kappa = 1$ 、 $p^* = 0.5$ ，對稱損失恰好正確；而本文的動作空間成本異質且 κ 由硬體決定，對稱損失因而失準。

做法。 我們採用兩種形式，並在第 6 節比較：

(L1) 動作成本加權的 L2。在 $\log$ (距下次存取時間) 的迴歸上，令每個樣本的權重等於「策略對該 block 所採取的動作，在判斷錯誤時的代價」：

$$\mathcal{L}_{\text{L1}} = \sum_i w(a_i) \cdot (\hat{y}_i - \log \tau_i)^2, \quad w(a_i) = \frac{\text{cost}_{\text{err}}(a_i)}{\text{cost}_{\text{err}}(\text{Cpu})} \quad (10)$$

此形式是 LRB 的損失加上其不需要的那一項，在 LightGBM 中僅需 sample_weight，無額外機制。

但必須注意一個校準上的限制。對連續目標的迴歸加權，改變的是條件期望 $\mathbb{E}[\log \tau | x]$ ，而非決策門檻。(L1) 本身因而不保證產生式 (9) 的最佳邊界。我們將 (L1) 定位為啟發式，並在其後串接一個明確的門檻校準步驟：於驗證集上將預測值映射為 $\hat{p}$ （例如以 isotonic regression），再依式 (9) 設定各層級的門檻。

對稱訓練加事後移動門檻是本文的基線。 上述步驟指向一個明顯的替代方案：以對稱損失訓練，僅在決策時將門檻自 0.5 移至 p^* 。此即 Elkan [12] 的結果，且具備一個實際優點—— κ 隨平台改變時無需重新訓練。若本文的增益全部來自門檻移動，則加權訓練並無貢獻。我們因此在表 15 的 (B) 區塊納入該基線（「對稱 L2 @ p^* 」一列），使兩者可分。

我們預期加權訓練仍有增益，其理由如下。Elkan 的結果成立於機率估計在整個值域上等品質的前提；有限容量的模型並不滿足此前提，它必須決定將擬合能力配置於何處。對稱損失依樣本密度均勻配置，而決策邊界 $p^* = 1/(1 + \kappa)$ 隨 κ 增大而深入機率分佈的尾端：於 MI300X / 8B ($\kappa = 6$) 為 0.14，於平台 A / 70B ($\kappa = 190$) 則為 0.005。在後者，對稱損失幾乎沒有誘因把該區域學準——該處樣本稀少且對平均平方誤差的貢獻極低。加權訓練的作用即是將擬合能力重新配置至該區域。

此論證給出一個可證偽的預測：(B) 區塊中「對稱 L2 @ p^* 」與 (L1)/(L2) 的差距，應隨 κ 增大而擴大。若兩者在 3090 與 MI300X 上差距相同，則加權訓練的正當性不成立，本文將據此縮減主張至門檻校準。

(L2) 動作成本進入排序損失的 **gain** 項 (原則性版本)。PARROT [29] 是我們調查中唯一刻意使用非對稱損失的工作，其 NDCG 形式明確地「重罰把機率放在即將被重用的 line 上，輕罰放在遠期重用的 line 上」，並帶來 3.5% 的命中率提升。然而 PARROT 只有單一動作，其 **gain** 項 ($d(l_w) - 1$) 僅編碼重用距離，未編碼動作成本。我們將動作成本置入該 **gain** 項：

$$\text{DCG} = \sum_w \frac{\text{cost}_{\text{err}}(a_w) \cdot (d(l_w) - 1)}{\log(\text{pos}(l_w) + 1)} \quad (11)$$

與 Sibyl 的差異在於成本是否進入訓練。Sibyl [40] 是唯一處理異質層級放置且考慮成本的系統，但它迴避了此問題：其 reward 直接取自實測延遲 $1/L_t$ ，成本比是硬體量出來的而非猜的。這是優雅的做法，但 Sibyl 的動作可逆 (頁面可在下次存取時重新放置)，而本文的 Drop $\rightarrow$ 重算不可逆，因此需要 Sibyl 的驅逐懲罰項 $R_p = 0.001 \times L_e$ 推廣為 per-action 懲罰。

硬體條件化。 $w(a_i)$ 由 FLOP/byte 比值決定，而該比值在 RTX 3090 為 2,254、MI300X 為 20,752 (表 4)。同一個損失函數在兩個平台上會產生不同的權重，進而學出不同的策略。第 2.6 節的硬體條件化主張，在此獲得一個具體且可量測的機制，而非僅是敘述。

此處須明確一項取舍。加權訓練使模型綁定於特定的 κ ，平台變更即需重訓；上一段的兩階段做法則保持平台無關，但擬合能力的配置不隨決策邊界移動。二者並非優劣關係而是不同的操作點：前者以可移植性換取邊界附近的精度，後者反之。(B) 區塊同時量測二者，使該取舍成為可報告的結果而非未言明的假設。

5.4 推論開銷預算

既有工作給出三種可行的攤銷方式：KVP [34] 於 prefill 後評分一次、decode 期間零額外 FLOP (10K 上下文下 0.71 ms，相對於 404 ms 的 prefill)；ForesightKV [11] 每 256 個新 token 重新評分一次，開銷 2.7%；TRIM-KV [8] 於 block 建立時評分一次，之後以指數衰減 β^{t-i} 免費地重新排序。

我們無法直接採用 TRIM-KV 的「建立時評分一次」，因為第 5.2 節的特徵 (deltas 與 EDC 計數器) 在每次存取時更新，一次性評分會立即過時。我們改採三層攤銷：(i) 存取時以極低成本更新 EDC；(ii) 於驅逐時機依 LRB 的做法取樣 $k = 64$ 個候選評分而非排序整個 cache，該操作在 CPU 上約需 30 μs ；(iii) 粗粒度的週期性全量重新評分。

線上學習控制器能否符合真實硬體預算，Pythia [7] 提供了存在性證據：其線上 RL 硬體 prefetcher 僅佔 1.03% 的晶片面積。

5.5 與現有機制的關係

Tiara 不取代壓縮或稀疏注意力，而是決定何時對哪個 block 施加何種處理。其與 vLLM 的 paged KV 佈局相容，並可與 LMCache 的分層後端並存。

6 Evaluation

本節報告平台 A 上已完成的量測。每一個數字皆可追溯至一條指令與一個輸出檔，追溯表見附錄 A；凡尚未量測者一律標為 NOT_MEASURED，不以推算值替代。尚未執行的實驗設計 (消融矩陣、平台 B 的移植量測) 列於附錄 B，其結果不進入本節的任何主張。

§6.1 給出設定，§6.2 給出產生本節全部 headroom 數字的 oracle 構造，§6.3–§6.9 給出七項量測結果，§6.10 依預先訂定的判準給出 go/no-go 判定。

6.1 設定

平台 A：NVIDIA RTX 3090 (24 GB GDDR6X, PCIe 4.0 $\times$ 16、sm_86)。本節全部量測皆在此平台完成；軟體版本與量測協定見附錄 A。第 2.6 節設計的平台 B (MI300X) 尚無機器，其實驗設計列於附錄 B。

本文的主張以成本常數 κ 、 P^* 、 α 表述，而非以特定加速器表述：換一張卡，常數改變而判別式的形式不變 (§6.8 以縮放後的 α 給出四個算力級距的預測)。文中「平台 A」指表 12 的該組常數。真正只在 sm_86 成立的結果——§6.6 的 FP8 後端限制——標為硬體相依。

模型剖面。我們量測表 1 的三個剖面。兩個模型的 κ 相差 2 倍，提供同一硬體上的第二個資料點；同一模型的 BF16 與 AWQ 則隔離出權重精度的影響。三組成本常數並列於表 12，本節每一處引用皆標明用的是哪一組。

工作負載。生產 trace 取自 Mooncake [36] 的 toolagent (23,608 請求) 與 conversation (12,031 請求)，中位長度 6,352 token、最長 126,208。目標長度區間 (65K–524K) 無公開 trace 可用，以合成成長上下文補足，其重用率與長度分布由我們設定，此限制於 §6.10 明確標註。

Baselines。我們量測五個現成系統：full_gpu (不卸載，vLLM 預設)、cpu_lru 與 cpu_arc (vLLM OffloadingConnector 的兩個內建策略)、tier_fs (CPU 加磁碟兩階)、以及 LMCache [2]。lru 與 arc 構成 vLLM 目前 production 預設策略的完整集合，其餘策略須經 CachePolicy 介面自行註冊。二者皆僅依存取歷史決策：一個「重算需 225 μs 」的 block 與一個「自 CPU 取回需 4.2 μs 」的 block 在 LRU 眼中完全等價，而本文的成本模型正是針對此空缺。學術與學習式對照 (InfiniGen、KIVI、KVP、ForesightKV、LookaheadKV、稀疏檢索一系) 尚未執行，其選取理由與可行性評估見附錄 B。

可推廣性的界線。本文只處理單請求的放置決策。多租戶下的機會成本、以及並行度造成的容量壓力，與此正交且未涵蓋；本文亦不提出能耗結論 (皆見第 7 節)。

6.2 Oracle：headroom 的量測工具

本節其後每一個 headroom 數字都來自同一個離線 oracle，故先給出其構造。oracle 取得整條 trace 的完整未來存取序列，據此構造一個離線放置策略 (演算法 2)。GPU 階的逐出用 Bélády/MIN，單階時此規則可證明最優 [6]。被逐出的 block 需再選一個目的地，我們跑兩條規則並取較佳者：cascade 無條件往下推，cost-aware 只在該階成本低於丟棄的邊際成本時才放。兩條都是可實作的離線策略，取其較佳者仍然可實作。Fang 等人 [14] 已為兩

Algorithm 2 離線 oracle (單趟重放；兩條目的地規則各跑一次取較佳者)

Require: trace T 、成本模型 C 、各階容量 cap_ℓ 、規則 $d \in \{\text{CASCADE, COST-AWARE}\}$

- 1: $\text{nxt}[b] \leftarrow b$ 的所有未來存取索引, $\forall b$
- 2: $\text{tail}[r, p] \leftarrow \sum_{k \geq p} C_{\text{Drop}}(k \cdot B) \triangleright$ 前綴語意下丟棄的邊際成本
- 3: $\mathcal{G}, \mathcal{C}, \mathcal{S} \leftarrow \emptyset \triangleright$ GPU、CPU、SSD 階
- 4: **for** 每個請求 $r \in T$ **do**
- 5: $g \leftarrow$ 第一個未命中的位置 $\triangleright$ vLLM 回傳 token 數，缺口後全數重算
- 6: **for** 每個位置 p 與其 block b **do**
- 7: **if** $p > g$ **then**
- 8: $\text{cost} += C_{\text{Drop}}(p \cdot B)$
- 9: **else**
- 10: $\text{cost} += C_{\text{tier}(b)}(p \cdot B) \triangleright$ 依 b 所在階計價
- 11: **end if**
- 12: $\mathcal{G} \leftarrow \mathcal{G} \cup \{b\}$
- 13: **while** $|\mathcal{G}| > \text{cap}_{\text{gpu}}$ **do**
- 14: $v \leftarrow \arg \max_{b \in \mathcal{G}} \text{nextuse}(b) \triangleright$ Bélády/MIN
- 15: $\mathcal{G} \leftarrow \mathcal{G} \setminus \{v\}$
- 16: **if** $\text{nextuse}(v) = \infty$ **then**
- 17: **continue** $\triangleright$ 不再用到，丟棄成本 0
- 18: **end if**
- 19: $\delta \leftarrow \text{tail}[\text{nextuse}(v)] \triangleright$ 丟棄 v 的邊際成本
- 20: $\text{ok}_\ell \leftarrow (d = \text{CASCADE}) \vee (C_\ell < \delta), \ell \in \{\mathcal{C}, \mathcal{S}\}$
- 21: 依序試 $\mathcal{C}, \mathcal{S}$ ：可放則放；該階已滿則與其中最不划算者交換，被擠出者續試下一階；皆不可則丟棄 v
- 22: **end while**
- 23: **end for**
- 24: **end for**

層 (HBM 與 off-package DRAM) 的無損放置推導上界；本文的 oracle 在兩點上延伸之：納入 SSD 層，以及納入 Drop $\rightarrow$ 重算動作。

oracle 付的價與線上策略相同。 其一，重算按 block 的絕對位置計價 (式 (2))；若讓 oracle 一律以位置 0 的價格重算，它付的價會低於線上策略，等於 oracle 自己作弊。其二，前綴語意下丟棄一個 block 的邊際成本是它到請求結尾的整條尾巴 (演算法 2 第 2 行) 而非它自己那一塊。其三，逐出去向逐項計數：若「不再用到、丟棄成本為零」的逐出 (第 16–17 行) 佔滿全部，則 CPU 與 SSD 階對最優解毫無貢獻，量到的 headroom 全部來自 GPU 階內的逐出選擇，不足以支持多階動作空間。

所報 headroom 為下界。 多階且動作成本異質的最佳放置我們並未求解。第 20–21 行的貪婪選擇不保證全域最優，且逐出未做前綴感知——真正的最優解會刻意讓保留的 block 構成連續前綴以避免製造缺口，而本 oracle 一樣會被缺口懲罰。兩項都使其偏弱，故所報 headroom 為下界，真正的最優只會更高。此不對稱使 GO 判定安全，NO-GO 判定則須額外審視。與 Sibyl [40] 報告「達到全知 oracle 的 80%」的評估傳統一致，我們以此上界作為判準而非成績。

求解範圍限於四個無損動作。 oracle 只在 Gpu_{16} 、 Cpu 、 Ssd 、 Drop 上求解，未納入三個精度階。這四個動作於

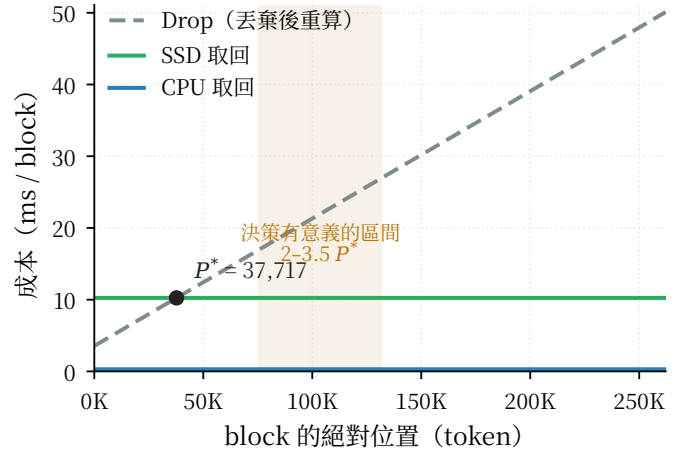


Figure 4: 三個動作的成本結構 (qwen-awq、NVMe)。Cpu 與 Ssd 的取回成本與 block 位置無關，Drop 則線性成長，二者於 $P^* = 37,717$ token 交叉。底色區為 $2-3.5 P^*$ ，即 P^* 落在請求內部、同一請求的不同位置需要不同決策的長度區間 (§6.8)。

§6.6 已驗證為逐字元無損，故 ϵ 約束在此自動滿足，本節的 headroom 是位置分層本身在 $\epsilon = 0$ 下可得的空間。精度階能額外帶來多少空間，本文未量。實作、複雜度與不變量檢查見附錄 A.3。

6.3 動作空間的有效子集由成本常數決定，可事先算出

動作空間 $\mathcal{A}$ 的六個狀態並非在每個平台上都存在，也並非在每個上下文長度下都有意義。其有效子集由兩個量決定，二者皆自成本常數 (表 12) 算出，不需試跑。

精度階由硬體與後端支援決定。 Gpu_8 是儲存狀態而非運算狀態，故不受 tensor core 是否原生支援 FP8 所限；我們實測四個 GPU 精度階皆為 `--kv-cache-dtype` 的取值，無須額外實作 (§6.9)。但可切換不等於可用：§6.6 顯示三個低精度階中僅 INT8 一階在檢索任務上保住品質。同一個動作在不同硬體上是否存在，本身即隨平台而變。

位置階由 P^* 與請求長度的關係決定。 Cpu 與 Ssd 的取回成本與位置無關，而 Drop 隨 block 的絕對位置線性成長 (式 (2))，48 個量測點、擬合偏差 $< 1.6\%$ 。二者於 P^* 交叉：其前 Drop 較廉，其後 Ssd 較廉。 P^* 隨成本剖面變動 $3.5\times$ (表 12)，故於 128K 上下文落在 Drop 一側的 block 佔 8.3% (llama-bf16) 至 28.8% (qwen-awq)。

Takeaway. 六階動作空間是候選集合，不是每個平台上都成立的必要集合。這不是本文主張的反例，而是其可操作形式：兩個判別式都只需要成本常數，因此「該用哪些動作」可以在部署前算出，而非試出來。

6.4 Tier 0 baselines：長上下文下五者有四者失效

我們以兩輪相同長前綴的工作負載量測五個現成系統：第一輪 (cold) 無任何快取，第二輪 (warm) 量其保留能力。二者的比值即為該系統卸載機制的價值。

Table 8: Tier 0 baselines 的 warm 加速比 (qwen-awq、4 個前綴、平台 A)。ctx 32K–65K 時五者相近 (42–77×)，因工作集仍可置入 GPU，卸載階未被使用。

對照組	機制	cold (ms)	warm (ms)	加速
<i>ctx = 131,072</i>				
full_gpu	不卸載	99,681	100,152	1.0×
cpu_arc	CPU 階 + ARC	102,702	72,883	1.4×
tier_fs	CPU + 磁碟兩階	101,518	13,626	7.5×
lmcache	LMCache	102,248	102,603	1.0×
<i>ctx = 258,048</i>				
full_gpu	不卸載	325,520	322,579	1.0×
cpu_arc	CPU 階 + ARC	327,295	325,767	1.0×
tier_fs	CPU + 磁碟兩階	326,393	43,664	7.5×
lmcache	LMCache	333,046	335,034	1.0×

於 258K，五個系統中僅 tier_fs 保有效益 (省下 282.7 秒)，其餘四者的 warm 與 cold 幾無差異。CPU 階於 131K 尚有 1.4×，至 258K 即失效：24 GiB 的 CPU 階可容 449,536 token，而 4 個 258K 前綴的工作集為 1,032,192 token。

一項保留。lmcache 於全部長度皆為 1.0×。此可能為我們的組態錯誤而非該系統的能力上限；在排除此可能之前，本文不宜稱 LMCache 於長上下文無效。

6.5 decode 佔總時間的四分之三，而放置決策對其無自由度

放置決策僅能作用於 prefill。decode 期間該請求的 KV 必須整份駐留於 GPU：每一步皆需讀取完整 KV，無搬移至 CPU/SSD 或丟棄重算的餘地。

以 Mooncake trace 的真實 output_length (toolagent 中位 30、p90 507；conversation 中位 350、p90 597) 與實測的每步成本換算，decode 佔端到端時間的 48.8% (toolagent) 與 53.8% (conversation)。於 llama-bf16 剖面 (BF16 權重 15.2 GB) 此比例升至 75%，因每一步的固定成本包含讀取權重。

$$t_{\text{decode/step}} = \beta + \gamma \cdot N_{\text{blocks}} \quad (12)$$

實測擬合 ($R^2 = 0.999$)：llama-bf16 為 $36.768 + 0.005581N$ ，qwen-awq 為 $18.158 + 0.001267N$ ms。 γ 換算之 KV 讀取頻寬分別為 376 與 581 GB/s，皆低於 RTX 3090 的 936 GB/s 峰值，構成一項物理合理性檢查。

此項使端到端 headroom 約為 prefill headroom 的 0.88 倍。於真實 trace 上，qwen-awq 的端到端 headroom 為 8.42% (toolagent) 與 9.23% (conversation)，落於 §6 標準的 5–15% 區間。但 Mooncake 的請求長度中位數僅 **6,352 token**，遠低於本文目標區間；於 65K–262K 的長度上端到端 headroom 為 17%–29% (§6.8)。

Table 9: 同一組 KV 精度於四種任務型態的分數 (qwen-awq、平台 A)。FP8 為 vLLM 的靜態未校正縮放，INT8/INT4 為 per_token_head 動態縮放。GSM8K 為 $n = 1,000$ 的 many-shot 推理；大海撈針為 32K 上下文、5 個深度 × 4 次重複的字串檢索；LongBench 為 7 個英文任務 ($n = 50$ 、32K 視窗、官方指標) 的巨觀平均；RULER 為 7 個合成任務 ($n = 30$ 、16K) 的巨觀平均。逐任務分數見表 13。末列的全距是本節的全部內容：同一組設定，換個任務型態， ϵ 差 21 至 36 倍。

KV 精度	容量	GSM8K 推理	大海撈針 檢索	LongBench 理解	RULER 整合
BF16	1.00×	77.9	100.0	58.4	91.6
FP8	2.00×	76.2	5.0	6.6	3.3
INT8	1.94×	76.4	95.0	57.0	79.1
INT4	3.77×	75.1	0.0	0.4	0.0
全距 (pp)	—	2.8	100.0	58.0	91.6

6.6 品質約束的作用範圍：位置分層無損，退化是「精度 × 任務」的性質

ϵ 約束對動作空間中的兩類動作意義不同，量法亦須不同。

無損動作： $\epsilon = 0$ 是可驗證的斷言。CPU 與 SSD 僅搬移位元組，理應不改變輸出。此為正確性斷言而非品質取捨：若輸出有異，即為實作錯誤。我們以逐字元比對驗證之——以 64-shot GSM8K (共用前綴 11,029 token) 於平台 A 對 60 題取輸出的 SHA-1，與 GPU-BF16 基準比對：

設定	正確率	逐字元相同
full_gpu (基準)	76.67%	—
cpu_lru	76.67%	60/60
tier_fs (CPU+ 磁碟)	76.67%	60/60

位置分層的三階在 ϵ 約束下等價； ϵ 的實質內容全部落在精度分層上。

有損動作：退化由任務決定，不由精度決定。GSM8K 上四者的差異均與零無法區分 ($n = 1,000$ 時差值的 95% 信賴區間為 ± 3.7 個百分點)；同一組設定於檢索任務上由 100% 降至 0%。此落差不是某個上下文長度的巧合：於 4K、8K、16K 三個長度上，BF16 皆為 100%，INT8 為 95/90/90%，而 FP8 為 5/0/5%、INT4 為 0/0/0% (表 13)。僅以推理任務評測會得出「KV 量化幾無代價」的錯誤結論。

功效分析顯示 GSM8K 無法用於量測 $\epsilon(f)$ 曲線：區分 2.8 個百分點需每設定 $n \approx 1,760$ ，7 個 f 值即 12,320 次請求，而 2.8 個百分點已是 $f = 1$ 的上界效果。

真實文件上的長上下文理解與檢索同側，不與推理同側。大海撈針是合成的單鍵字串比對，其崩潰可能是該任務的特性而非長上下文能力的特性。我們以 LongBench 的 7 個英文任務檢驗此可能：涵蓋單文件 QA、多跳 QA、摘要、few-shot 分類與合成檢索，每任務取前 50 筆，依上游 pred.py 的協定 (過長者自中間截斷、few-shot 任務不套 chat template) 於 32K 視窗量測，計分用上

游 `metrics.py` 的指標。7 個任務無一例外地站在檢索那一側：巨觀平均自 BF16 的 58.4 降至 FP8 的 6.6 與 INT4 的 0.4。逐題配對後，FP8 掉 51.8 個百分點 (95% bootstrap CI [47.3, 56.3])、INT4 掉 58.0 [53.7, 62.3]。摘要 (GovReport, ROUGE-L) 與多跳 QA (HotpotQA、2WikiMQA, F1) —— 兩個不需要逐字回收任何字符串的任務 —— 的掉幅與純檢索同量級。

「哪個精度安全」的答案取決於評測選了哪個 benchmark。INT8 在前三項評測上都讀起來無損：GSM8K -1.5 個百分點 (n.s.)、LongBench -1.3 [0.0, 2.8]、大海撈針 -5。但於 RULER 的 7 個合成任務 (16K、 $n=30$) 其巨觀平均為 79.1 對 BF16 的 91.6，配對差值 12.5 [8.6, 16.7] —— 與零可區分。差距幾乎全部來自單一任務：niah_multikey_3，即鍵與值皆為 UUID、且整片 haystack 由同構的干擾針構成的變體，INT8 於此只剩 53.3 (配對差值 46.7 [30.0, 63.3])。同一個精度階在四個評測上分別是「無損」「無損」「幾乎無損」與「掉一半」。因此不能以單一標量寫進部署設定，式 (14) 取最差任務保留率而非跨任務平均，其必要性由這一系列數字直接證成。

梯度由「答案是否為某個確切位置的確切 token」決定。RULER 的任務族張開了這條軸，而退化沿軸單調。答案是全域統計量者最耐受：FWE (Zipf 分佈下最高頻的三個詞) 是 14 個任務中唯一 FP8 未全失者 (21.1)，亦是唯一 INT8 未掉者 (87.8 對 BF16 的 85.6，差值 -2.2 [-6.7, 2.2])。需要聚合但必須逐項列出者次之：CWE (前 10 高頻詞) 掉 12.0 [4.7, 20.3]。多跳追蹤 (VT) 掉 6.7 [-2.0, 16.0]。而四個 NIAH 變體 —— 答案是某個確切位置的確切字符串 —— 掉 6.7 至 46.7。KV 量化的雜訊會抹掉逐 token 的位置證據，卻不會抹掉整份上下文的統計形狀。

主因為格式而非縮放方式。數值分析顯示：FP8 靜態縮放的相對誤差為 2.64%，改用 per-token-head 動態縮放僅降至 2.56%，而 INT8 動態縮放為 0.65%。FP8 e4m3 的 3 個尾數位元將相對精度鎖定於約 12.5%，縮放方式無法改變；INT8 於指定範圍內有 255 個刻度。理論值 (FP8 12.5%/√12 = 3.6%、INT8 3/440 = 0.68%) 與實測吻合。數值誤差並可單調預測檢索結果：0.65%→95%、2.64%→5%、11.76%→0%。

於 sm_86，動態縮放的 FP8 不可用。fp8_per_token_head 需 Triton attention backend，該後端於 compute capability 8.6 上拒絕 FP8 KV cache。因此平台 A 的精度階實際僅有一級可用 (BF16 → INT8)。此為 κ 主張的另一實例：同一動作在不同硬體上是否存在，本身即隨平台而變。

6.7 寫入頻寬是磁碟階的可行性門檻，而成本感知放置將其降低 4–7 倍

模擬的成本模型僅對「取回」計價，而寫入亦消耗頻寬。我們以 trace 的真實時長換算各策略所需的持續磁碟寫入，結果見表 10。

Oracle 所需的寫入頻寬是 tier_fs 的 1/4.0 至 1/7.1。此比值在兩個剖面、兩個 trace 上皆成立，是本節唯一不

Table 10: 各策略所需的持續磁碟寫入頻寬 (MiB/s，512 GiB 磁碟階、Mooncake trace)。兩個模型剖面的 KV 每 token 位元組數相差 2.3×，故寫入需求隨剖面等比縮放。

剖面	trace	最佳 baseline	tier_fs	Oracle
llama-bf16	toolagent	tier_fs	4,666	1,114
llama-bf16	conversation	tier_fs	4,892	1,214
qwen-awq	toolagent	cpu_arc	2,016	283
qwen-awq	conversation	cpu_arc	2,122	330

隨剖面改變的結果：成本感知的放置決策在達成表 10 所在設定的 headroom 時，對磁碟的寫入壓力低於逐層下推的 tier_fs 一個接近數量級的幅度。

兩個剖面的可部署性結論不同。於 llama-bf16，tier_fs 既是最佳 baseline，又需要 4,666–4,892 MiB/s，超出唯一做過持續寫入量測的裝置 (SATA QLC，181 MiB/s) 26×，亦超出 NVMe 的短測上界 2,512 MiB/s。即使以「每個不重複 block 僅寫一次」的下界計算仍需 3,086–3,208 MiB/s，故此結論不依賴 load-and-evict 的建模選擇。於 qwen-awq (本文主設定)，最佳 baseline 為 cpu_arc，它完全不使用磁碟階；tier_fs 的需求降至 2,016–2,122 MiB/s，其下界為 1,350–1,404 MiB/s。

我們不宜稱磁碟階於 NVMe 上不可部署。該判定需要 NVMe 的持續寫入量測，而 §A.2 記錄我們沒有做。可宣稱者為兩點：寫入頻寬是磁碟階一道獨立於延遲的可行性約束，而 §6.1 的五個 baseline 無一將其納入設計；成本感知放置把這道約束放寬 4–7 倍。

6.8 headroom 的峰值出現在成本交叉點的 2–3.5 倍處

放置決策的價值不隨上下文長度單調成長，而是在特定長度見頂。此現象由三個動作的成本結構決定：CPU 與 SSD 的成本與位置無關，而 Drop 的成本隨 block 的絕對位置線性成長 (式 2)。二者的交叉點

$$P^* = \frac{C_{\text{ssd}} - C_{\text{recompute}}^0}{\alpha} \quad (13)$$

將請求分為兩段：位置在 P^* 之前的 block 重算較便宜，之後的 block 搬移較便宜。當請求長度遠小於 P^* 時所有 block 皆應重算，遠大於時所有 block 皆應搬移；兩種情形下放置策略均無選擇餘地。只有當 P^* 落在請求內部，同一請求的不同位置才需要不同決策。

我們固定記憶體壓力為 5×、重用率 80.9%，僅改變請求長度 (GPU 預算隨工作集等比放大)。平台 A 上 qwen-awq 的 $P^* = 37,717$ token (表 12)，量測結果見圖 5：headroom 於 32K 僅 8.94% (MARGINAL)，於 65K–262K 為 24.44%–33.50% (三者皆 GO)，於 524K 降至 12.71% (MARGINAL)。峰值落在 $3.5P^*$ 處。524K 一列另有兩項限制：單請求的 KV 超出 GPU 預算，且該長度的上半段 (位置 258,048 以上) 超出 qwen-awq 成本模型的擬合上限，為線性外插。

峰值位置隨硬體移動，此為 κ 主張的直接推論。式 13 的分母 α 由 prefill 吞吐決定。加速器越快則 α 越小、 P^* 越大、峰值長度越長。以實測的 α 按倍率縮放模擬 (圖

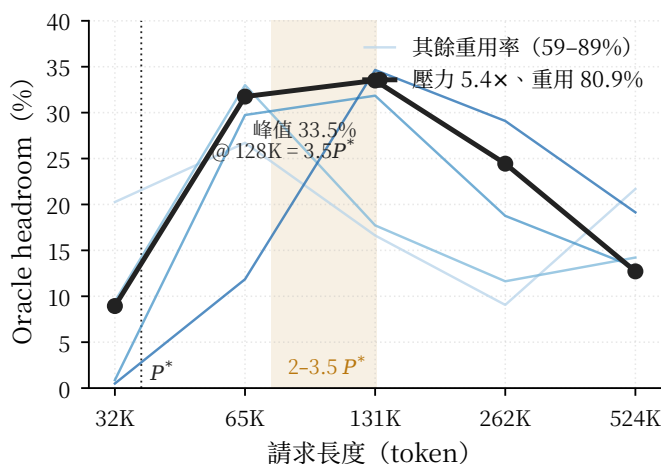


Figure 5: Oracle headroom 隨請求長度的變化 (qwen-awq、NVMe、僅計 prefill)。粗線為固定壓力 (5.4 $\times$) 與重用率 (80.9%) 的掃描，細線為其餘三個重用率 (59%–89%) 下的量測。四條線的峰值皆落在 2–3.5 P^* 的底色區內，故峰值位置由成本結構決定而非由重用率決定。

6)，峰值自平台 A 的 256K 移至 6 倍算力下的 512K，且峰值 headroom 自 18.7% 升至 39.5%。其政策意涵為同一套放置策略不可能同時適用於不同加速器，即本文第 2 節主張的量化形式。

於 MI300X，峰值落在 1M 級的單請求上下文。以表 4 的 BF16 峰值算力比 (1,307.4/71.2 = 18.4 $\times$) 外推，該平台的 P^* 為 346K–693K (視 MFU 而定)，2–3.5 P^* 的甜蜜點因而落在 692K–2.4M token。1M 級的單請求上下文正落於此區間之內，而 Llama-3.1-8B 於 1M 需 122 GB KV，單張 MI300X 可容納 (表 3)。此預測因此在一張卡上就能證實或推翻，不需要多租戶設定。

同一組數字也界定了生產工作負載的位置：Mooncake 的最長請求為 126,208 token，僅為該峰值的 1/10 至 1/19，故今日的生產流量在此類硬體上量不到放置決策的價值——這不是方法的缺陷，而是 P^* 隨算力右移的直接推論。若於 1M 量到的 headroom 顯著低於平台 A 峰值的量級，本節的機制解釋即被推翻。

以上三段為可證偽的預測。 α 的縮放假設 prefill 時間與峰值算力成反比，忽略了 MFU 隨模型與序列長度的變化、kernel 實作差異、以及 ROCm 與 CUDA 的成熟度落差。驗證方式明確：於目標加速器上重量 §A.2 的四項成本常數 (CPU、SSD、 $C_{\text{recompute}}^0$ 、 α)，代入式 13 求 P^* ，再以本節的方法掃描長度。若峰值不落在 2–3.5 P^* ，本節的機制解釋即被推翻。

6.9 權重精度決定 24 GB 級加速器能否進入長上下文區間

Gpu₈ 是儲存狀態，不需要 FP8 tensor core。Ampere (sm₈₆) 確無原生 FP8 tensor core 運算，但本文動作空間中的 GPU-FP8 是儲存狀態——KV 以 FP8 存放、讀取時反量化——與 tensor core 無關。我們在平台 A 上實測 --kv-cache-dtype fp8 可正常運作：Llama-3.1-8B

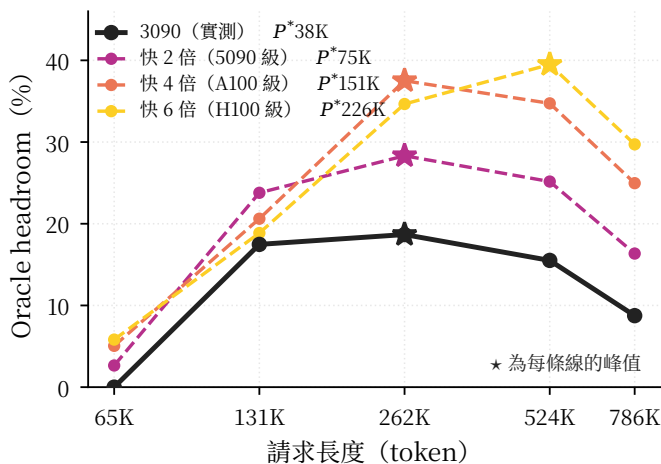


Figure 6: headroom 的峰值隨加速器算力往更長的上下文移動 (壓力 5 $\times$ 、重用率 80.9%、僅計 prefill)。★ 為每條線的峰值：自平台 A 的 262K 移至 6 $\times$ 算力下的 524K，峰值 headroom 自 18.7% 升至 39.5%。實線為平台 A 的實測；虛線條將實測的 α 按算力倍率縮放，為推算而非量測，其驗證方式見下段。

的 KV 容量由 41,648 增至 83,312 token、Qwen2.5-7B-1M 由 106,512 增至 213,040 token，二者皆為恰好兩倍且位元組佔用不變。

更進一步，vLLM 亦提供 int8_per_token_head 與 int4_per_token_head，故動作空間中位於 GPU 上的四個精度階，在平台 A 上皆為同一旗標的不同取值，無須額外實作。以每 GiB 可容 token 數正規化後 (消除 KV pool 大小的 run-to-run 抖動，全距降至 0.04%) 實測得：

KV dtype	KiB/tok	理想	額外	vs BF16
auto (BF16)	128.11	128	+0.11	1.00 $\times$
fp8	64.04	64	+0.04	2.00$\times$
int8_per_tok_head	66.04	64	+2.04	1.94 $\times$
int4_per_tok_head	34.02	32	+2.02	3.77$\times$

fp8 為純格式轉換故恰為 2 $\times$ ；兩個 per-token-head 量化階則各固定多出 2 KiB/token。此開銷可由架構直接算出：Llama-3.1-8B 有 32 層、8 個 KV head，K 與 V 各一份，故每 token 有 $2 \times 32 \times 8 = 512$ 個 scale，以 FP32 儲存即 $512 \times 4 = 2,048$ bytes = 2 KiB，與實測相符 (int8 為 4.1 bytes/scale、int4 為 4.0)。故精度階梯的收益並非 2 $\times$ /4 $\times$ ，而是 2.00 $\times$ /1.94 $\times$ /3.77 $\times$ ；式 (1) 若僅以每元素位元組數計算，將系統性高估低精度階的收益。量化中繼資料是與序列長度成正比的固定成本，必須進入成本模型。

BF16 權重在 24 GB 卡上佔 15 GB，KV 預算僅餘 5.9 GB，容量上限 41,648 token。此配置無法進入本文關心的 128K 區間。AWQ-INT4 權重佔 4.7 GB，KV 預算增至約 15 GB：

設定	單卡 KV 容量 (token)
Llama-8B BF16 權重	41,648
Llama-8B AWQ 權重	120,320
Llama-8B AWQ + FP8-KV	240,656
Qwen-7B-1M AWQ 權重	273,872
Qwen-7B-1M AWQ + FP8-KV	547,744
DeepSeek-V2-Lite MLA AWQ	399,376

Table 11: go/no-go 判定材料。前四列為 Mooncake 真實 trace（中位長度 6,352 token），末列為目標長度區間的合成長上下文。所有設定：平台 A、NVMe 磁碟階、512 GiB。

剖面	工作負載	計時	headroom	對停止條件
qwen-awq	toolagent	端到端	8.42%	不觸發
qwen-awq	conversation	端到端	9.23%	不觸發
llama-bf16	toolagent	端到端	2.83%	觸發
llama-bf16	conversation	端到端	3.34%	觸發
qwen-awq	65K–131K	prefill	31.8–33.5%	不觸發

Takeaway. 在正確的權重配置下，24 GB 消費級 GPU 的 KV 容量不是長上下文的瓶頸。BF16 權重僅餘 41,648 token，而 AWQ-INT4 權重加 FP8-KV 使同一張卡達 240,656 (Llama-8B) 與 547,744 token (Qwen-7B-1M)，相差 13.2 $\times$ 。此點對本文的雙平台設計有兩個後果。其一，平台 A 足以驗證 128K 區間的容量與成本主張，無須更大的加速器。其二，MLA 架構將 KV 壓至 30.4 KiB/token (GQA 的 1/4.2)，使單卡容量達 399,376 token，即在同一張 24 GB 卡上把壓力軸往後推了 4.2 $\times$ ——與換一張大卡的效果同向。這使 MLA 成為 κ 軸上的第三個資料點，且不需要第二台機器。

6.10 go/no-go 判定：條件式 GO，且條件可事先算出

停止條件。 若 oracle 相對於最佳可部署策略的改善幅度小於 5%，我們終止此研究方向並報告此負面結果。此判準訂於任何策略實作之前，本節依其判定。

判定材料。 表 11 列出五組設定的 headroom。判定隨權重精度與請求長度改變，而兩者的機制本節前文皆已量到。

判定：條件式 GO。 我們不終止此方向，並將主張限縮為條件式。兩個條件各有已量到的機制。其一，**權重須量化**：BF16 權重使 decode 佔端到端時間的 75%，而放置決策對 decode 無自由度 (§6.5)，故 llama-bf16 的 headroom 落在停止條件之下；AWQ-INT4 權重使 decode 的固定成本減半，headroom 隨之升至 8–9%。其二，**請求長度須落在 2–3.5 P^*** ：Mooncake 的中位長度 6,352 token 僅為 P^* 的 0.17 倍，遠在甜蜜點之外；於 65K–131K，同一剖面的 prefill headroom 為 31.8%–33.5% (§6.8)。兩個條件皆可由表 12 的成本常數事先算出，無須試跑。

判定的適用範圍。 表 11 的五列相差 12 $\times$ ，因此 headroom 是一條隨剖面與長度變動的曲線，取其中任一列為代表都會誤導；本文引用時一律標明設定。三處尚無數據：線上策略能取得 oracle 的多少比例，需策略實作後才可量；平台 B 的常數需要機器；而末列 65K–131K 為合成長上下文，其重用率與長度分布由我們設定。目前沒有公開 trace 同時具備長上下文與真實的低重用率，這是本文最實質的證據缺口 (§7)。

7 Discussion and Limitations

線上策略尚未實作，故本文不報告端到端效能。這是本稿最主要的限制。第 6 節以 oracle 量到了改善空間的上界，但取得該上界的策略尚未存在。**oracle 是上界不是成績**。以 Sibyl [40] 報告的「達成全知 oracle 的 80%」為參照，本文的上界換算為生產 trace 上 6.7–7.4%、甜蜜點 25–27% 的實際改善。此換算取自單一先前系統的達成率，不是本文的量測，列出僅為說明上界與可交付效能之間的距離。

ϵ 是估計值而非保證。品質約束以 profiling 建立的代理訊號執行， $\Delta Q \leq \epsilon$ 是經驗性的估計而非形式保證。這是所有品質約束工作的共同限制。將其提升為執行期可驗證的界限（例如以 per-head 的注意力分佈失真上界）是自然的後續方向，但其理論難度高。

重算可能不划算。 第 2.4 節的分析顯示重算的單次成本比傳輸高 6–190 倍。若 p_i 的預測精度不足，Drop 動作可能持續造成淨損失。附錄 B 的消融設計將直接檢驗此點；若 Drop 不帶來收益，我們將自系統中移除之。此檢驗尚未執行。

K 與 V 未分離處理。 式 (1) 前導的 2 對應 key 與 value，本文將二者綁定於同一動作：一個 block 的 K 與 V 一同降級、一同量化、一同丟棄。這是刻意的範圍限制，而文獻指出該粒度並非最佳。KIVI [31] 顯示二者的元素分佈不同，key cache 宜沿 channel 維度量化而 value cache 宜沿 token 維度量化；Hariri 等人 [20] 進一步指出在固定記憶體預算下，將位元優先配置給 key（如 4-bit key 搭配 2-bit value）較均勻配置嚴格更優；KVtuner [27] 則於逐層粒度搜尋混合精度，在 Llama-3.1-8B-Instruct 上達成 3.25 位元的近無損配置。

因此本文階梯上的 Gpu_8 與 Gpu_4 應理解為 K/V 共用精度的保守設定，其品質退化為 K/V 分離配置的上界。將動作空間沿 K/V 拆為兩軸是自然的延伸，但會使 $|A|$ 自 6 增至 36，並使標籤生成與 oracle 求解的成本相應上升。本文不涵蓋此延伸；它與本文的核心主張正交——無論動作以何種粒度定義，「動作成本異質時損失函數須編碼該成本」的論點皆成立，且動作粒度變細只會使成本的離散程度更高，因而強化而非削弱該論點。

ROCm 生態的成熟度。 平台 B 的移植風險不在本文的策略，而在其下的分層機制：ROCm 上的 KV offload 路徑在 CI 覆蓋、失敗處理與 attention 後端選取上皆不及 CUDA，逐項狀態見附錄 B.6。此風險影響的是平台 B 實驗何時可執行，不影響本文已量到的常數。

與 AMD 生產堆疊的關係。 AMD 的 ROCm/mori 中的 MORI-UMBP 提供「具分層儲存與分散式鍵值存取的統一記憶體與頻寬池」，已整合入 SGLang HiCache 並具備 SSD 層。我們不宣稱 AMD 平台上缺乏分層 KV 基礎設施；本文的貢獻在於其放置策略，而非分層機制本身。

平台 A (RTX 3090) 的限制。消費級顯卡缺乏記憶體與計算分軌的能耗計數器，故本文不提出任何能耗結論。MI300X 的 `amdsmi` 具備 HBM 與計算分軌的累加器，是做此分析的正確平台，但我們尚無該機器。

學習式對照組未釋出權重。 KVP [34]、ForesightKV [11] 與 LookaheadKV [4] 三者均未釋出訓練後的權重，且其預測器皆為模型專屬——KVP 需為 Qwen2.5-7B-Instruct 的 112 個 layer-head 各訓練一個 agent (其 README 載明使用 8 GPU DDP)，ForesightKV 需至少兩張 GPU 分置訓練模型與參考模型，LookaheadKV 僅提供「minimal training example」。重訓因而是每個主模型各一次的完整訓練，且主模型變更時須全部重做。

我們因此不以外部學習式系統作為核心主張的檢驗手段。本文的主張是「對稱損失在異質動作成本下失準」，而檢驗該主張的正確方式是在同一系統內替換損失函數 (表 15 的 (B) 區塊)，此設計消除了系統間的無關差異。外部學習式 baseline 僅用於佐證整體競爭力，其數值取自原論文並明確標註未重跑。

無法重跑的對照組。 ArkVale、Quest、ShadowKV、ClusterKV、RetroInfer 與 ParisKV 因 `rapidsai/raft` 無 ROCm 移植而無法於平台 B 執行；我們於平台 A 執行之，但無法提供這些方法在 MI300X 上的數值。

階梯假設主機記憶體是獨立的一階。 在 CPU 與 GPU 共用同一塊 HBM 的 APU 上 (如 MI300A [3, 5])，「卸載至 CPU 記憶體」僅是同一記憶體內的頁表變更，不產生容量收益，階梯塌縮為 HBM → 對等 APU HBM → NVMe。本文的成本模型形式不變，但 Cpu 一階須重新標價。

8 Conclusion

本文的核心論點是：當動作成本相差 6–190 倍、且該比值本身隨硬體變動 32 倍時，以對稱損失訓練的效用預測器會系統性地錯置決策邊界。我們將 KV 放置形式化為受品質約束的序列決策問題，其動作空間統一了位置分層與精度分層，並將「丟棄後重算」視為階梯上一個具明確標價的層級。

平台 A 的量測支持此論點 (第 6.3 節)。其中兩項確立了 GPU 精度階的可用性：FP8 為儲存狀態而非運算狀態，故 `sm_86` 可量測，且動作空間中位於 GPU 的四個精度階皆為同一旗標的取值。另兩項確立了成本模型的形式：per-token-head 量化固定多帶 2 KiB/token 的中繼資料，使精度階梯的收益為 $2.00 \times 1.94 \times 3.77 \times$ 而非 $2 \times 4 \times$ ；而固定大小 block 的重算成本對其絕對位置為線性，非二次。

第四處修正的性質不同。SSD 的取回成本為常數，而 Drop 的成本隨 block 的絕對位置線性成長 (式 (2))，二者於 P^* 交叉：其前 Drop 較廉，其後 SSD 較廉。 P^* 本身隨模型剖面變動 $3.5 \times$ (表 12)，故落在 Drop 一側的 block 比例於 128K 上下文為 8.3% (llama-bf16) 至 28.8% (qwen-awq)。這不是本文主張的反例，而是其直接實例：最佳動作空間的有效子集由上下文長度與成本剖面共同決定，而二者皆可事先算出。六階動作空間應理解為候選集合。

量測亦劃清了品質約束的作用範圍。CPU 與 SSD 的輸出與 GPU-BF16 逐字元相同 (60/60)，故位置分層的三階在 ϵ 約束下等價； ϵ 的實質內容全部落在精度分層上。以 $n = 1,000$ 的 GSM8K 量測，四個精度階的差異仍與零無法區分 (全距 2.8 個百分點)，但同一組設定於大海撈針檢索上由 100% 降至 0%、於 LongBench 的 7 個真實文件任務上自 58.4 降至 0.4、於 RULER 的 7 個合成任務上自 91.6 降至 0.0 (§6.6)。更尖銳的是 INT8：它在前三者上讀起來無損，卻在 RULER 的 UUID 多鍵檢索上只剩 53.3。 ϵ 是「精度 $\times$ 任務」的函數，單一任務量不出它，而「哪個精度安全」的答案取決於評測選了哪個 benchmark。

尚未完成的工作有三項。其一，策略的實作：go/no-go 判定已完成 (§6.10，條件式 GO)，但策略本身尚未存在。其二，平台 B 的數據：本文的 κ 跨硬體主張目前只有單平台的實證 (同一台機器上 P^* 隨剖面變動 $3.5 \times$)，跨加速器的部分仍為推算。其三，一份同時具備長上下文與真實低重用率的 trace：Mooncake 的中位長度僅 6,352 token，而 SCBench 的重用率高達 80%，兩者皆非目標情境。第 A.3 節給出的求解方法效力範圍約束了這些後續工作可以宣稱什麼。

Statements

Data Availability. 第 6 節的全部量測、產生它們的指令、以及成本常數的原始量測檔，將與量測工具鏈一併公開釋出。每一列結果均帶有可反查原始輸出的識別碼。所引用模型組態取自公開的 HuggingFace `config.json`。線上策略尚未實作，故無對應的程式碼可釋出。

Ethics. 本研究不涉及人類受試者或個人資料。工作負載將使用公開 benchmark 與已匿名化的公開對話 trace。

Author Contributions (CRediT). (投稿時填入) Conceptualization; Methodology; Software; Investigation; Writing — original draft.

Conflict of Interest. (投稿時填入)

Funding. (投稿時填入)

AI Usage Disclosure. 本稿的文獻調查、比對分析與初稿撰寫過程使用了 AI 輔助研究工具。所有引用文獻均經作者查證其存在性與內容；所有量化分析均可由公開規格與模型組態重現。AI 工具未用於產生實驗數據。

A 平台 A 的量測明細

本附錄給出第 6.3 節所引數字的量測協定與逐設定數據。所有量測於單張 RTX 3090 (`sm_86`、driver 550.163.01、CUDA 12.4) 上以 vLLM 0.28.0 搭配 PyTorch 2.13.0+cu129 執行，每列數據可由 `run_id` 追溯至指令與輸出檔。

A.1 量測衛生

該機器由二十餘位使用者共用，此事實決定了三項協定。

Table 12: 平台 A 的成本常數 (ms/block)。本文所有 oracle 與掃描結果的磁碟階皆為 NVMe；SATA 一欄僅供裝置對照，不用於任何結論。 $P^* = (C_{\text{ssd}} - C_{\text{recompute}}^0)/\alpha$ 為 SSD 與 Drop 的交叉點。qwen-awq 的重算量測涵蓋 11 個位置，故位置擬合上限遠高於 llama-bf16 的 5 個位置。

	qwen-awq NVMe (主設定)	llama-bf16 NVMe	llama-bf16 SATA (對照)
量測 ctx	96,000	16,384	16,384
Cpu	0.298	0.543	0.588
Ssd	10.245	6.286	5.536
Drop 基底	3.546	4.008	4.008
Drop 斜率 α	0.000178	0.000210	0.000210
P^* (token)	37,717	10,851	7,278
位置擬合上限	258,048	24,576	24,576

每卡獨佔不足以保證乾淨。GPU 的 SM 為各卡獨佔，但 PCIe、host RAM 頻寬與 `/dev/shm` 為全機共用。他人於其他卡上的負載不會出現在本卡的 `compute-apps` 中，卻會拖慢搬移量測。我們逐格比較序列執行與五卡並行執行的 80 個量測點：不經 PCIe 搬移 KV 的 `full_gpu` 差異在 $\pm 2\%$ 內，而經 PCIe 的 warm TTFT 差異達 -26% 至 -52% 。故每列數據記錄整機爭用等級，且僅序列模式的數據進入論文。

KV pool 大小在同一設定下並非常數。 同一模型、同一卡、同一 `gpu_memory_utilization`，GPU KV cache size 在 5.09 與 5.88 GiB 兩值間擺動（原始 token 數全距 13.5%）。以每 GiB 可容 token 數正規化後全距降至 0.04%，故容量的平時成本一律以 KiB/token 報告，而非原始 token 數。前一行程結束至驅動釋放記憶體之間存在延遲，我們要求連續三次取樣皆達標才啟動下一次量測。

分層量測須驗證資料確實落到目標層。量測 SSD 時若 CPU 階容量大於工作集，資料不會 cascade 至磁碟。我們的首次量測即因此失效：CPU 階 24 GiB、工作集 8 GiB，vLLM 的 `chunk_queries` 顯示磁碟階僅被查詢 2 次而 CPU 階 2,048 次，所得「SSD 與 CPU 僅差 1.2%」實為 CPU 階的數字。將 CPU 階縮至 1 GiB 後磁碟階被查詢 2,048 次，實際讀取 3 GiB。每列數據記錄兩層的 `chunk_queries`。

A.2 成本常數

平時成本。 以每 GiB 可容 token 數量測，每設定重複三次取中位數，全距 $\leq 0.04\%$ 。此正規化消除 KV pool 大小的 run-to-run 抖動（見 §A.1）；四個精度階的實測值見 §6.9 表列。

被需要時的成本：成本常數不可跨剖面替代。 我們以共用前綴的兩輪工作負載量測 warm TTFT，扣除 GPU 命中的基準後換算為每 block（16 token）的成本。成本常數隨模型剖面與磁碟裝置變動，故本文在每一處引用時標明用的是哪一組（表 12）。qwen-awq 於 NVMe 上的量測為主設定：CPU 2,651.7 ms、SSD 62,328.9 ms（96K 上下文、扣除 GPU 命中的 860.7 ms 基準），Drop 自位置 0 的 453.9 ms 成長至位置 258,048 的 6,320.2 ms。

qwen-awq 的取回量測只在 NVMe 上進行，故該剖面沒有 SATA 的對應值。原始量測與其來源指令隨論文一併釋出。

P^* 在固定裝置（NVMe）上換模型剖面即相差 $3.5\times$ ，換磁碟裝置再得 $1.5\times$ 。兩項皆發生在同一台機器上，未換加速器。這是 §2.4 之 κ 主張的一個單平台實例：動作成本的比值不只隨加速器世代變動，在固定加速器上也隨模型剖面與儲存裝置變動。

磁碟階的成本並非由 I/O 主導。 我們以 `O_DIRECT` 循序存取量測兩顆裝置，測試檔 1 GiB、各重複三次取中位數。SATA QLC (Samsung 870 QVO，`/ssd7`) 讀 522 MiB/s，NVMe (Crucial P3，`/`) 讀 2,085 MiB/s，相差 $4.0\times$ ；端到端 warm TTFT 卻是 5,803 (SATA) 與 6,600 ms (NVMe)，頻寬較高者反而較慢。以 warm TTFT 除以搬移的位元組數 ($16,384 \text{ token} \times 128.11 \text{ KiB} = 2,048 \text{ MiB}$) 得有效頻寬 353 (SATA) 與 311 MiB/s (NVMe)，僅達各自裸讀的 68% 與 15%，其餘為分層機制的查表、cascade 排程與 promotion 開銷。成本模型因此拆為 $C_{\text{ssd}} = C_{\text{lookup}} + C_{\text{sched}} + C_{\text{io}}(\text{device})$ ，僅末項與裝置有關。兩顆碟的有效頻寬僅差 $1.1\times$ 而裸讀差 $4.0\times$ ，此落差即為前兩項的量化證據：於 NVMe 上，磁碟階的成本有 85% 不是 I/O。

QLC 的持續寫入只有短測的 37%。SATA QLC 的 1 GiB 短測給出 492 MiB/s ($n = 3$)，16 GiB 的長測只剩 181 MiB/s ($n = 2$ ，兩次為 190 與 172)；短測整個落在 QLC 的 SLC 寫入快取內。KV 階持續寫入，可行性判定因此須採用 181 MiB/s。

同一台機器的 NVMe 於 1 GiB 短測寫 2,512 MiB/s，故兩顆碟在同一種測法下相差 $5.1\times$ 。我們未對 NVMe 執行 16 GiB 的持續寫入測試，因此其持續寫入能力為 **NOT MEASURED**。第 6.7 節凡引用 NVMe 寫入頻寬處，該數字皆為短測值，只能作為上界。此缺口是刻意標明而非疏漏：SATA 上短測高估 $2.7\times$ ，故短測值無法用於判定任何裝置的可部署性；補上此量測是接手工作的第一項（見 §7）。

SSD 這個動作的成本因此在單一台機器內就相差數倍，而表 5 的 κ 僅涵蓋加速器與主機之間的傳輸，未涵蓋磁碟階。

A.3 Oracle 的求解方法與其效力範圍

§6.10 給出 oracle 的構造與其為何是下界。本節補充實作與驗證。

實作。 oracle 先對整條 trace 建立每個 block 的未來存取索引表，再單趟重放。GPU 階以 max-heap 搭配 lazy deletion 取 Bélády 犧牲者，每次逐出 $O(\log n)$ ；先前的版本每次逐出掃描整個 GPU 集合，於 17,117 blocks 的預算與 20 萬次存取下約需 3.5×10^9 次比較而無法完成。CPU 階以 next-use 的 max-heap 保留下次使用最近者，SSD 階以「相對丟棄的節省量」min-heap 先換掉最不划算者，兩者同樣採 lazy deletion。被 CPU 階擠下來的 block 會再往 SSD 階試一次，而非直接丟棄。

不變量檢查。 模擬器在每次掃描前後自動執行不變量檢查。開跑前檢驗 trace 的單位、成本模型的外插範圍、以及磁碟階容量是否物理可行；跑完後檢驗三項：命中數守恒（各階命中與重算之和等於總存取數）、強制未命中下限（重算次數不得低於不重複 block 數）、以及 oracle 為上界（oracle 的總成本不得高於任一線上策略）。任一項失敗即代表模擬器有錯，結果不得採用。

第三項曾在 2026-08-31 失敗：SSD 階 2 TiB 時 oracle 的 headroom 為 -3.16%，依定義不可能。原因是成本感知規則以單一 block 而非整條尾巴估計丟棄成本，做出局部便宜、全域昂貴的決策。修正後全部通過。

效力前提。 其一，成本常數須為實測：模擬器不接受推估值，且要求 GPU 預算、KV 每 token 位元組數與成本常數三者取自同一個模型剖面，否則拒絕執行。其二，模擬器須能複現已量測的行為。我們以第 6 節的工作負載比對 full_gpu 相對 cpu_lru 的 warm 段成本比：16K 上下文實測 9.00 對模擬 9.73（差 8%），32K 上下文實測 12.27 對模擬 12.66（差 3%）。絕對值同樣相符，full_gpu 為 5,033 對 5,864 ms、cpu_lru 為 559 對 603 ms。驗證僅涵蓋一種工作負載形狀與兩個有鑑別力的上下文長度，本文引用 headroom 的絕對值時一併標明此範圍。

逐 block 的簡化對前綴結構的工作負載無害。 vLLM 的查詢語意是連續前綴：_lookup_complete_chunks 回傳一個 token 數，而 token 數無法表達「block 0、1、4、5 命中，2、3 未命中」這種形狀，因此第一個缺口之後的 block 一律重算。我們把模擬器改為此語意並量化其影響：缺口之後的 block 中仍留在任一階的比例為 0.000% 至 0.464%（兩條 trace × 三個策略），其中 61.8% 至 80.1% 是首次出現的 block，本來就必須重算。真實流量的重用本身即為前綴結構，Mooncake 的 hash_ids 就是前綴雜湊，第一個未命中因此恰好落在共用前綴的結尾，缺口之後沒有可供損失的快取內容。此結果同時構成模擬器的一項有效性證據。

A.4 RTX 3090 的 BF16 峰值如何得出

表 4 的 71.2 TFLOPS 由 328 TC × 1,695 MHz × 128 FLOP/clock 得出 [1]；GA10x 的 tensor core 於 FP32 累加下為 128 FLOP/clock（FP16 累加為 256），BF16 與 FP16-FP32 累加同速。此處易誤取 35.6 TFLOPS——該值對應 64 FLOP/clock，為 TF32 tensor 或非 tensor FP32 的峰值，與 BF16 GEMM 的資料路徑不同。FLOP/byte 欄為此值除以 31.5 GB/s。

A.5 長上下文品質評測的逐任務分數

表 9 的 LongBench 與 RULER 兩欄是巨觀平均。跨任務平均會把單一任務的崩潰稀釋掉，而 §6.6 的主張正是那些崩潰，故此處給出逐任務值。

信賴區間以配對 bootstrap 求得。 四個設定評的是同一批題目，故差值逐題相減後再重抽題目（10,000 次），而非把兩個獨立平均相減。配對消去題目難度的變異：於 LongBench 合併全部 350 題，INT8 的差值為 1.3 個百分

Table 13: 大海撈針、LongBench 與 RULER 的逐任務分數（qwen-awq、平台 A、分數為各任務的官方指標 × 100）。大海撈針一塊為 5 個深度 × 4 次重複，其餘為每任務前 n 筆。四個設定吃的是同一批 prompt（同題目、同順序、temperature=0、固定 seed），唯一變動的是 --kv-cache-dtype。巨觀平均為逐任務平均後再對任務取平均——F1 與 ROUGE 的尺度不同，不可對題目直接加權。**FP8 於這 14 個任務上最高只到 22.0、INT4 最高 1.5**：表中的零是量到的零，不是缺資料。

任務	類別	n	BF16	FP8	INT8	INT4
LongBench（真實文件、32K 視窗）						
MultiFieldQA	單文件 QA	50	51.5	7.4	51.3	0.2
Qasper	單文件 QA	50	41.4	2.8	40.8	0.5
HotpotQA	多跳 QA	50	53.4	2.2	54.1	0.0
2WikiMQA	多跳 QA	50	52.2	7.5	50.9	0.0
GovReport	摘要	50	34.0	2.1	34.1	1.5
TREC	分類	50	76.0	22.0	74.0	0.0
PassageRetr.	合成檢索	50	100.0	2.0	94.0	0.3
巨觀平均	—	—	58.4	6.6	57.0	0.4
大海撈針（合成、單鍵、逐上下文長度）						
4K	單鍵檢索	20	100.0	5.0	95.0	0.0
8K	單鍵檢索	20	100.0	0.0	90.0	0.0
16K	單鍵檢索	20	100.0	5.0	90.0	0.0
32K	單鍵檢索	20	100.0	5.0	95.0	0.0
RULER（合成、16K）						
NIAH multikey-2	多鍵檢索	30	100.0	0.0	90.0	0.0
NIAH multikey-3	多鍵（UUID）	30	100.0	0.0	53.3	0.0
NIAH multivalued	一鍵四值	30	100.0	0.0	93.3	0.0
NIAH multiquery	四鍵查詢	30	100.0	0.0	92.5	0.0
VT	多跳追蹤	30	83.3	1.3	76.7	0.0
CWE	詞頻（前 10）	30	72.3	0.7	60.3	0.0
FWE	詞頻（前 3）	30	85.6	21.1	87.8	0.0
巨觀平均	—	—	91.6	3.3	79.1	0.0

點，95% CI [0.0, 2.8]；未配對的版本寬約三倍。§6.6 引用的所有區間皆為此法所得。

此處的絕對值不可與公開榜單並列。 三項差異使然：模型是 Qwen2.5-7B-Instruct-1M 的 AWQ-INT4 權重、且移除 DCA 的變體（vLLM 0.28 的 V1 engine 無可用的 DCA 路徑）；每任務取前 50 筆而非全量 150–200 筆；視窗為 32K。但四個設定吃的是同一批 prompt，故跨精度的相對變化不受這三項影響——本文宣稱的正是相對變化。RULER 另有兩處與上游的刻意差異：haystack 一律用上游自身提供的 noise 選項（essay 版需自 paulgraham.com 取得，非可再散布），且不含需 SQuAD/HotpotQA 原始檔的 qa 類（真實文件的理解由 LongBench 該半負責）。前者影響 niah_multivalued 與 niah_multiquery：任務語意不變，難度略降。

計分函式已對上游逐筆驗證。 LongBench 上游的 metrics.py 依賴 jieba（僅中文任務需要），為免將其拉進推論環境，本文的英文四項指標（qa_f1、ROUGE-L、classification、retrieval）為重寫。重寫過的計分函式若與上游有出入，本節的數字便不是 LongBench 分數，故以 9,500 組隨機字串與真實預測對上游逐筆比對，容差 10^{-9} 下零筆不一致。

Table 14: 成本常數的量測協定。左欄為 block 閒置時持續付出的代價，右欄為被需要時一次性付出的代價。Drop 的價值完全來自左欄的零。

動作	平時成本（閒置）	被需要時的成本
Gpu ₁₆	GPU 位元組	≈ 0
Gpu ₈	GPU 位元組 /2	反量化 kernel
Gpu ₄	GPU 位元組 /4	反量化 kernel
Cpu	host 位元組	PCIe（無反量化）
Ssd	磁碟位元組	NVMe I/O + PCIe
Drop	0	重算 (ℓ)

B 尚未執行的實驗設計

本附錄收錄已設計但尚未執行的實驗。其內容不支持正文的任何主張；列出的目的是使設計可被檢視，並使「哪些證據還不存在」一目了然。

B.1 成本常數的量測協定

式 (7) 的每一項都需要一組平台與模型相關的常數。這些常數每個 (平台, 模型) 組合只量一次，之後由策略直接查用，不隨每次實驗重量。

關鍵在於兩類成本必須分開量測——這正是第 2.4 節論證的核心，若壓成單一維度便會得到「重算最貴故永不重算」的錯誤結論：

量測方式。 平時成本以 `torch.cuda.memory_stats()` 與 host 端計數器直接取得，單位為位元組，可由式 (1) 驗算。被需要時的成本則以隔離的微基準取得：對每個動作，反覆搬移或重建固定數量的 block 並取中位數延遲，重複三次確認變異小於 10%。

重算成本不是常數。 `recomp( $\ell$ )` 隨 block 的絕對位置 ℓ 成長。此協定設計時假設其為 attention 的二次項；§6.3 的量測顯示形式為線性。我們將其量測為位置的函數而非單一純量，並以線性層的 $2N_{\text{active}}$ 項作為 $\ell \rightarrow 0$ 的下界檢核。對 MoE 模型， N_{active} 為啟用專家的參數量而非總參數量。

B.2 消融設計

此為本文最重要的實驗。表 15 分為五個區塊，各回應一個不同的質疑。

(A) 四項成本回應「把四項成本組合起來是否構成貢獻」。消融以成本項為單位而非以模組為單位：若移除任一項均導致顯著退化，其必要性即為實證結果而非主張。我們亦承認相反的可能：若移除某項不造成退化，正確做法是將其自系統中移除，使貢獻更為聚焦。

(B) 損失函數是核心主張的直接檢驗。若成本敏感損失相對於對稱 L2 無可量測優勢，則第 5.3 節的整個論證不成立。此區塊不可省略。

(C) 機制為次要，確認品質約束與精度維度確有作用。

(D) 預測器檢驗第 5.2 節「選 GBDT 而非神經網路」的取捨：前三列共用攤平特徵，檢驗容量；GRU 一列直接消費 `deltas` 序列，檢驗結構。

(E) 決策粒度檢驗一個至今為預設值而非量測結論的參數，見下文。

兩平台必須分開報告。第 5.3 節論證 $w(a_i)$ 由 κ 決定，而 κ 在兩平台相差 32 倍（表 5）。(B) 區塊在兩平台上應呈現不同的退化幅度：在 κ 較大的 3090 上，對稱損失的代價應更明顯。若兩平台退化幅度相同，則硬體條件化的主張被否證——這是本文最關鍵的可證偽點。

最佳的決策粒度由 attention tile 的大小決定，故隨後端而變。式 (4) 的動作逐 block 施加（圖 3a，16 token），但這是預設值而非量測結論：§6.6 的四個精度階皆以 `--kv-cache-dtype` 量得，而該旗標作用於整個引擎，故所有 block 同一格式；§6.2 的 oracle 又未納入精度階。本文因而從未執行過混合精度的 attention。

令 g 為精度決策的粒度。其兩端各有代價： g 小則保留逐 block 調整的自由度，但 KV block 為 16 token 而 attention kernel 的 tile 典型為 64–128 token，格式邊界因而落在 tile 內部，同一次載入須以兩套佈局解讀——§6.9 量到的 2 KiB/token 量化中繼資料正是使兩種格式佈局不同的原因。 g 大則 tile 恆為單一格式，但冷熱 block 併入同一段後只能取折衷精度，容量收益隨之下降。兩者相互抵消，故存在中間的最佳點，其位置由 tile 大小 g_{tile} 決定。

此結構與 §6.8 的 P^* 同形：一個自常數算出的特徵尺度，界定策略的有效操作區間。 g_{tile} 因而是第五個須逐平台量測的常數，而 tile 幾何隨 attention 後端而變——§B.6 已記載啟用 KV connector 會使 vLLM 自候選清單移除 ROCm_ATTN 後端。(E) 區塊因此是「常數改變則最佳策略隨之改變」的第三個實例，前兩者為加速器世代（表 5）與模型剖面（表 12）。

此區塊於平台 A 不具鑑別力，須待平台 B。§6.6 顯示三個低精度階中僅 INT8 保住品質，且 `fp8_per_token_head` 所需的 Triton 後端於 `sm_86` 拒絕 FP8 KV cache，故平台 A 實際只有 BF16 與 INT8 兩階可選；兩階之下，逐 block 與逐段的差異無從展開。CDNA3 具原生 FP8，若 `fp8_per_token_head` 於 ROCm 上可用（此為 NOT_MEASURED，列為平台 B 到手後的第一項驗證），該平台方有三階以上。若三個粒度於兩平台上的差異均不顯著，則粒度不是有意義的設計維度，我們將據以把圖 3a 的作用範圍改以段表述，並將此軸降為實作細節。

B.3 權重精度的敏感度分析

權重精度不是本文的決策變數，但它改變 KV 可用的預算，因而間接影響最佳策略。我們將其列為敏感度分析而非主要結果：

平台	權重	目的
A (3090)	AWQ-INT4	主要設定（記憶體受限）
A (3090)	BF16	對照：量化對 KV 預算的影響
B (MI300X)	BF16	主要設定
B (MI300X)	FP8	對照：CDNA3 之後的常見配置

待檢驗的問題：在固定的硬體與模型下，最佳 KV 策略是否隨權重精度改變？我們預期會——因為權重精度決定 KV 預算，而預算改變會移動式 (8) 的容量約束。但這

Table 15: 消融矩陣。每次移除或替換一項，量測 quality-latency-memory Pareto 前緣的退化。(B) 區塊是本文核心主張的直接檢驗。[†](D) 區塊另行回報熟路徑推論延遲，因較大的預測器須在計入其自身延遲後才可比較。前三列共用攤平特徵，檢驗容量；GRU 一列直接消費 deltas 序列，檢驗結構。任一列在計入自身延遲後仍佔優，第 5.2 節的對應設計選擇即被推翻。[‡](E) 區塊另需 attention kernel 層級的延遲，因 tile 破碎會被端到端延遲吸收而無從歸因；其餘三軸沿用式 (14) 的 $Q(\pi)$ 、有效 KiB/token (含碎片化) 與 TTFT/TPOT。 $g = \infty$ 一列即 §6.6 的全域統一設定，作為 harness 的複現錨點。

變體	移除／替換	Pareto 退化	
		3090	MI300X
(A) 四項成本			
Tiara (完整)	—	基準	基準
– 未來效用	$p_i \rightarrow \text{recency}$	TBD	TBD
– 傳輸成本	$\lambda_x = 0$	TBD	TBD
– 重算成本	移除 Drop	TBD	TBD
– 記憶體成本	$\lambda_m = 0$	TBD	TBD
(B) 損失函數 (核心主張)			
對稱 L2 @ 0.5	$w \equiv 1$ ，門檻 0.5	TBD	TBD
對稱 L2 @ p^*	$w \equiv 1$ ，門檻式 (9)	TBD	TBD
(L1) 加權 + 校準	式 (9)	TBD	TBD
(L2) 成本進 NDCG	—	TBD	TBD
(C) 機制與特徵			
– 品質約束	$\epsilon \rightarrow$ 固定預算	TBD	TBD
– 壓縮	移除精度維度	TBD	TBD
– 模型內部特徵	僅存取歷史	TBD	TBD
– query 側訊號	移除 attn_mass	TBD	TBD
(D) 預測器容量與結構 [†]			
GBDT (本文)	—	基準	基準
MLP 2×64	容量：同特徵、同損失	TBD	TBD
MLP 4×256	容量：同特徵、同損失	TBD	TBD
GRU (序列)	結構：直接吃 deltas	TBD	TBD
(E) 精度的決策粒度 ^{g†}			
$g = 16$ (預設)	邊界落於 tile 內	TBD	TBD
$g \approx 128$	對齊 tile，邊界重合	TBD	TBD
$g = 512$	對齊 hash_ids	TBD	TBD
$g = \infty$ (全域)	無邊界；錨點	TBD	TBD

是間接影響，與第 2.6 節的 κ 軸 (直接改變成本比) 性質不同。

B.4 平台 B (MI300X) 的實驗設計

平台 B (壓力軸的遠端)：AMD Instinct MI300X (gfx942, 192 GB HBM3)，ROCm 7.x，vLLM ROCm wheel (Python 3.12)，LMCache ROCm wheel(gfx942)，真實 NVMe SSD。此平台承擔並行度／機會成本場景與能耗分析。

壓力軸。 壓力軸隨平台而不同，這正是第 2.6 節雙平台設計的直接後果。

平台 A (RTX 3090, 24 GB)：既有文獻的「單請求、上下文遞增」設定在此成立。以 8-bit 權重的 Llama-3.1-8B 計，64K 上下文可置入而 128K 超出，主壓力軸為 context 8K→16K→32K→64K→ (128K，預期失敗)。

Table 16: 選模結果。 M 為 MoE。 κ 於各自平台計算。平台 A 的兩個模型 κ 相差 2 倍，提供同一硬體上的第二個資料點。

平台	模型	κ	角色
A (3090)	Qwen2.5-7B-Inst-1M	108	主力 (懸崖在 315K)
	Llama-3.1-8B-Inst	54	對照 (κ 減半)
B (MI300X)	Llama-4-Scout ^M	8	規模 (原生 10M)
	Qwen3-30B-A3B ^M	3	MoE 對 κ 的影響

平台 B (MI300X, 192 GB)：同一設定在此不成立：8B–14B 模型於 128K 僅佔 HBM 的 8–10% (第 2 節)，量測不到壓力。此平台改採三軸：

1. 上下文長度 (主軸)：128K、256K、512K、1M 的單請求上下文。
2. 模型架構：見下方選模準則。
3. 模型規模：8B 至 70B，檢驗 κ 隨模型增大而上升。

選模準則。 我們不以「涵蓋愈多模型愈好」為原則，而採一條更嚴格的規則：僅納入會使 κ 產生可事先計算之變化的模型。據此，選模受一個硬條件約束：**必須在目標平台上留有足夠的 KV 預算**。這排除了在平台 A 上使用 MoE：Qwen3-30B-A3B 即使以 int4 權重載入亦需約 17 GB，僅餘 4.6 GB 供 KV 使用。

據此，兩個平台的模型分工如下：

Qwen2.5-7B-Instruct-1M 為平台 A 的主力，因其原生支援 1M 上下文，而其 24 GB 上的容量懸崖落在約 315K token——遠低於模型上限，故量測到的是純粹的記憶體限制而非模型限制。相對地，Llama-3.1-8B 在同平台的容量懸崖 (約 132K) 與其原生上限 (131,072) 幾乎重合，兩種限制無法分離，故僅作對照。

明確排除的架構。 MLA (如 DeepSeek 系列) 改變式 (1) 本身，其 KV 為壓縮後的潛在向量；hybrid SSM (如 Jamba) 的狀態大小不隨序列成長，已由 Marconi [35] 處理。二者並非「未評估」，而是定義上不同的問題。兩個平台的壓力軸不同並非實驗設計的瑕疵，而是待檢驗的假設本身：若最佳策略確實隨硬體比值改變，則產生壓力的方式本來就應該不同。

B.5 尚未執行的 baseline 對照

表 17 給出 baseline 集合。此處須區分兩種用途不同的對照；混淆二者會使核心主張的檢驗失去判別力。

(i) **檢驗核心主張者為消融，而非外部系統。** 本文主張既有學習式方法所用的對稱損失在異質動作成本下失準。直接比較 Tiara 與 KVP 並不能檢驗此主張：二者在儲存層數 (六階對單階)、預測器族 (GBDT 對強化學習) 與特徵集上皆不同，量到的差異無法歸因至損失函數。具判別力的實驗是固定系統、僅替換損失函數，即表 15 的 (B) 區塊，其中「對稱 L2」一列即為既有方法的損失置於本文動作空間下的行為。該實驗完全在我們控制之下，不依賴任何外部訓練產物。

(ii) **外部學習式系統用於佐證整體競爭力。** KVP、Fore-sightKV 與 LookaheadKV 均已釋出程式碼，但未釋出訓

Table 17: Baseline 分層與平台可用性。平台 A (CUDA) 上全部可直接執行；平台 B 欄為移植至 ROCm 的估計工作量，經逐一檢視 repo 後得出。[†] 稀疏檢索一系因 rapidsai/raft 無 ROCm 移植而結構性地無法於平台 B 執行——這是平台 A 存在的理由之一。^{*} LMCACHE 在 ROCm 上雖有官方 wheel，但存在已知執行期問題，見第 7 節。ClusterKV、RetroInfer、ParisKV 同屬該系，因與上列三者機制高度重疊而未納入。

層級	Baseline	A (CUDA)	B (ROCm)
錨點	Full GPU (無卸載)	✓	✓
	Oracle (離線最優)	✓	✓
Production	vLLM lru	✓	✓
	vLLM arc	✓	✓
	vLLM + fs 磁碟層	✓	✓
	LMCache [2]	✓	✓*
學術 (系統)	InfiniGen [25]	✓	~1 週
	KIVI [31]	✓	3-7 天
學習式	KVP [34]	✓	3-7 天
	ForesightKV [11]	✓	3-7 天
	LookaheadKV [4]	✓	3-7 天
稀疏檢索 [†]	ArkVale [10]	✓	×
	Quest [43]	✓	×
	ShadowKV [42]	✓	×

練後的權重，且其預測器為模型專屬（第 7 節）。我們於平台 A 重訓可得者並回報，未能重訓者引用原論文數值並明確標註未重跑。此列的完整程度不影響 (i) 的結論。

(iii) **production 策略界定實用價值。** vLLM 的 OffloadingConnector 內建策略僅有 lru 與 arc，其餘須經 CachePolicy 介面自行註冊；此二者因而構成目前 production 預設策略的完整集合。兩者皆僅依存取歷史決策：以表 5 的 3090 / Llama-3.1-8B 為例，一個「重算需 225 μ s」的 block 與一個「自 CPU 取回需 4.2 μ s」的 block 在 LRU 眼中完全等價。本文的成本模型正是針對此空缺。

無法於 ROCm 執行者：ArkVale、Quest、ShadowKV、ClusterKV、RetroInfer、ParisKV。其阻擋原因並非 Flash-Infer (AMD 已維護 ROCm/flashinfer 移植)，而是 rapidsai/raft 無 ROCm 版本，以及 CUTLASS 的 template 層級耦合。我們將引用其論文回報數值作間接對照，並於第 7 節誠實說明。

B.6 尚未執行的評估指標

系統指標：TTFT、TPOT、端到端延遲、GPU/CPU/SSD 佔用、PCIe traffic、throughput、服務層級分佈（見下）、重算成本。

ROCm 工具鏈的已知狀態。 LMCACHE 無自動化的 on-device ROCm CI；其預設 MP 模式在 ROCm 上存在已知的死鎖問題；vLLM 在 ROCm 上的 KV 傳輸錯誤目前會導致 worker 終止而非降級（相關修正 PR 尚未合併）。此外，啟用 KV connector 會使 vLLM 自候選清單移除 ROCM_ATTN 後端，我們將記錄每次實驗實際使用的 attention 後端。

ROCm 量測方法。 KV 搬移經由 SDMA copy engine 而非 AQL compute queue，對 rocprof-compute 的 TCC/L2 counter 不可見。我們將使用 rocprofv3 --memory-copy-trace 取得逐筆傳輸的位元組數、方向與時間戳，並以 rocprof-compute 的 gfx942 "Remote Read Traffic" 指標取得 kernel 側視角。

能耗。 MI300X 的 amdsmi 提供 per-rail 能耗累加器，其中 hbm_energy_acc 將 HBM 能耗與計算能耗分離。對一個探討「保留於記憶體或重新計算」的研究問題而言，這是直接對應的量測能力，且在 NVIDIA 平台上不可得。

品質指標。 Yandex 的研究 [45] 顯示 ShadowKV 等方法在 needle-in-a-haystack 上表現良好卻在多事實抽取任務上失效。因此我們將 needle-in-a-haystack 降級為 sanity check：LongBench 與 RULER 已於 §6.6 量測完成（逐任務分數見 §A.5），§A.5 的 niah_multivalued 與 niah_multiquery 即多事實抽取的合成版本。**仍未量測者為真實長文本摘要在放置策略下的表現——**本文已量的是 KV 精度對摘要的影響 (GovReport)，而放置策略 (GPU / SSD / Drop) 在 §6.6 已驗證為逐字元無損，故該欄的空缺不影響本文主張，只影響未來的策略比較表。

品質如何化約為單一數字。 評估表的 Quality 欄須為單一純量，其定義如下。設 $\mathcal{T}$ 為上述任務集合， $Q_t(\pi)$ 為策略 π 於任務 t 的原始分數，並以 Full GPU 為參照定義相對保留率 $r_t(\pi) = Q_t(\pi) / Q_t(\text{FullGPU})$ 。我們回報

$$Q(\pi) = \min_{t \in \mathcal{T}} r_t(\pi), \quad (14)$$

即最差任務的保留率，而非跨任務平均。理由與上一段相同：平均會使單一任務的崩潰被其餘任務的分數掩蓋，而 [45] 所報告的正是此類崩潰。品質約束因而具體化為 $Q(\pi) \geq 1 - \epsilon$ 。逐任務的 r_t 完整列於附錄，避免式 (14) 的化約隱藏任務間的差異。

以服務層級分佈取代單一命中率。 「KV hit rate」對單層快取有明確意義，但本文的動作空間有六階，「命中」並非二元事件：自 CPU 取回與自 SSD 取回的成本相差約一個數量級，與重算則相差 6-190 倍（表 5）。將這些情形壓成單一比率會抹除本文所要量測的成本結構。我們因此回報服務層級分佈，即每次 KV 存取由 $\mathcal{A}$ 中哪一階滿足的經驗分佈，並另外給出兩個純量：(i) **GPU 常駐率**，存取時該 block 已位於 GPU 且為 BF16 的比例（對應傳統命中率）；(ii) **重算率**，由 Drop 觸發重算的存取比例。後者為本文特有，直接對應第 2.4 節的成本模型，亦是表 15 中「移除 Drop」一系列的解讀依據。

預測器層級的診斷：為何不回報分類準確度。 我們刻意不以預測器的分類準確度作為評估指標，並在此說明理由，以免被誤讀為疏漏。成本敏感訓練的作用即是將決策邊界自 0.5 移至 $p^* = 1/(1 + \kappa)$ ，其必然後果是模型在廉價方向上多犯錯以換取在昂貴方向上少犯錯。以未加權的準確度衡量，成本敏感模型應當劣於對稱模型；若其未劣化，反而意味著加權未生效。準確度因而與本

文的目標函數方向相反，不構成有效證據。我們改以兩個量診斷預測器：

(i) 校準品質。式 (9) 的最佳門檻僅在 $\hat{p}_i$ 為校準過的機率時成立：若模型輸出 $\hat{p} = 0.1$ 的樣本中實際重用比例並非約 10%，該門檻即失去意義，第 5.3 節的整個論證隨之失效。我們回報 isotonic regression 校準前後的可靠度圖與 expected calibration error。此量同時是第 5.3 節所揭露之 L1 缺陷（加權迴歸改變條件均值而非決策門檻）是否被校準步驟補救的唯一直接證據；缺少此圖，該缺陷等同未被處理。

(ii) 成本加權錯誤率。 $\sum_i c(a_i^{\text{pred}}, a_i^*)$ ，其中 a_i^* 為 oracle 動作、 $c(\cdot, \cdot)$ 為式 (7) 的成本函數。此量計的是「錯掉多少微秒」而非「錯了幾次」，是唯一能在預測器層級公平比較兩種損失函數的指標，亦與系統層級的端到端延遲直接可對照。

跨工作負載泛化。 承第 5.2 節的分布飄移討論，我們額外回報一組交叉實驗：於單一工作負載訓練並於其餘工作負載測試，回報上述兩個診斷量與 $Q(\pi)$ 的退化幅度。此結果同時界定週期性重訓的必要性——若跨工作負載退化可忽略，重訓機制即為不必要的複雜度，我們將據以移除。

References

- [1] NVIDIA ampere GA102 GPU architecture. Whitepaper, NVIDIA Corporation, 2021. v2.1; Appendix A. GeForce RTX 3090: 328 3rd-gen Tensor Cores, 1,695 MHz boost, 71 TFLOPS FP16/BF16 with FP32 accumulate (dense).
- [2] LMCACHE: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025. [PREPRINT]; <https://github.com/LMCACHE/LMCACHE>.
- [3] Advanced Micro Devices, Inc. AMD CDNA 3 architecture white paper. Technical report, Advanced Micro Devices, Inc., 2023.
- [4] Jinwoo Ahn, Ingyu Seong, Akhil Kedia, Junhan Kim, Hyemi Jang, Kangwook Lee, and Yongkweon Jeon. LookaheadKV: Fast and accurate KV cache eviction by glimpsing into the future without generation. In *International Conference on Learning Representations (ICLR)*, 2026. Samsung; arXiv:2603.10899.
- [5] Anonymous (see arXiv:2508.12743). Dissecting CPU-GPU unified physical memory on AMD MI300A APUs. *arXiv preprint arXiv:2508.12743*, 2025. [PREPRINT] measurement study.
- [6] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [7] Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021. arXiv:2109.12021.
- [8] N. Bui, S. Sharma, S. Lamba, S. Mishra, et al. Cache what lasts: Token retention for memory-bounded KV cache in LLMs. *arXiv preprint arXiv:2512.03324*, 2025. [PREPRINT] JPMorganChase AI Research + Yale; venue UNVERIFIED.
- [9] Celestial AI. AMD MI300X GPU performance analysis. *arXiv preprint arXiv:2510.27583*, 2025. [PREPRINT] Celestial AI.
- [10] Renze Chen, Zhuofeng Wang, Beiquan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. ArkVale: Efficient generative LLM inference with recallable key-value eviction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Peking University; Infinigence-AI.
- [11] Zican Dong, Peiyu Liu, Junyi Li, Zhipeng Chen, Han Peng, Shuo Wang, and Wayne Xin Zhao. ForesightKV: Optimizing KV cache eviction for reasoning models by learning long-term contribution. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03203.
- [12] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978, 2001.
- [13] Shaokai Fang, Ziang Li, Wenfei Wu, Jiatong Ji, Qingsong Liu, and Ruizhi Pu. Not all tokens are worth caching: Learning semantic-aware eviction for LLM prefix caches. *arXiv preprint arXiv:2605.18825*, 2026. [PREPRINT].
- [14] Yunhua Fang, Rui Xie, Asad Ul Haq, Linsen Ma, Kaoutar El Maghraoui, Naigang Wang, Meng Wang, Liu Liu, and Tong Zhang. Accelerating LLM inference via dynamic KV cache placement in heterogeneous memory system. *IEEE Computer Architecture Letters*, 2025. arXiv:2508.13231.
- [15] Shaoting Feng et al. AdaptCache: KV cache native storage hierarchy for low-delay and high-quality language model serving. In *Proceedings of the SOSP Workshop on Big Memory (BigMem)*, 2025. Workshop paper; arXiv:2509.00105.
- [16] Shaoting Feng, Yuhang Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. EvicPress: Joint KV-cache compression and eviction for efficient LLM serving. *arXiv preprint arXiv:2512.14946*, 2025. [PREPRINT] submitted 2025-12-16.

- [17] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*, 2024. arXiv:2403.19708.
- [18] Shiwei Gao, Youmin Chen, and Jiwu Shu. Fast state restoration in LLM serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2410.05004.
- [19] Ayushman Garg, Akshita Gupta, Shaswata Bhattacharya, Abhishek Gupta, Sandeep Kumar, and Manoj Kumar. QEvcit: Recoverable quantized KV eviction for attention-drift-robust long-context decoding. *arXiv preprint arXiv:2608.05326*, 2026. [PREPRINT] submitted 2026-08-05.
- [20] Mohsen Hariri, Alan Luo, Weicong Chen, Shaochen Zhong, Tianyi Zhang, Qifan Wang, Xia Hu, Xiaotian Han, and Vipin Chaudhary. Quantize what counts: More for keys, less for values. *arXiv preprint arXiv:2502.15075*, 2025. [PREPRINT] 2025-02-20.
- [21] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annamaram. KVPR: Efficient LLM inference with I/O-aware KV cache partial recomputation. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2025. arXiv:2411.17089.
- [22] Jiantong Jiang, Peiyu Yang, Rui Zhang, and Feng Liu. Towards efficient large language model serving: A survey on system-aware KV cache optimization. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2026. arXiv:2607.08057.
- [23] Shuwei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. Compute or load KV cache? why not both? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. PMLR v267; arXiv:2410.03065.
- [24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [25] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. Seoul National University; arXiv:2406.19707.
- [26] LeoAM authors. Breaking the boundaries of long-context LLM inference: Adaptive KV management on a single commodity GPU. *arXiv preprint arXiv:2506.20187*, 2025. [PREPRINT] under review.
- [27] Xing Li, Zeyu Xing, Yiming Li, Linping Qu, Hui-Ling Zhen, Wulong Liu, Yiwu Yao, Sinno Jialin Pan, and Mingxuan Yuan. KVtuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.04420.
- [28] Jian Lin, Jiazhi Mi, Zicong Hong, Haodong Wang, Qianli Liu, Haodyue Zhang, Peng Li, and Song Guo. KV-Drive: A holistic multi-tier KV cache management system for long-context LLM inference. *arXiv preprint arXiv:2605.18071*, 2026. [PREPRINT] submitted 2026-05-18; venue attribution unconfirmed.
- [29] Evan Z. Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. PMLR v119.
- [30] Zedong Liu, Xinyang Ma, Dejun Luo, Hairui Zhao, Bing Lu, Wenjing Huang, Yida Gu, Xingchen Liu, Zheng Wei, Jinyang Liu, Dingwen Tao, and Guangming Tan. KVserve: Service-aware KV cache compression for communication-efficient disaggregated LLM serving. In *Proceedings of the ACM SIGCOMM Conference*, 2026. arXiv:2605.13734.
- [31] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.
- [32] Xinyue Ma, Heelim Hong, Taegeon Um, Jongseop Lee, Seoyeong Choy, Woo-Yeon Lee, and Myeongjae Jeon. OrbitFlow: SLO-aware long-context LLM serving with fine-grained KV cache reconfiguration. *Proceedings of the VLDB Endowment (PVLDB)*, 19:1046, 2026. arXiv:2601.10729.
- [33] William Meng, Benjamin Lee, and Hong Wang. Understanding bottlenecks for efficiently serving LLM inference with KV offloading. *arXiv preprint arXiv:2601.19910*, 2026. MLSys 2026 per arXiv comments.
- [34] Luca Moschella, Laura Manduchi, and Ozan Sener. Learning to evict from key-value cache. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. Apple; arXiv:2602.10238.
- [35] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. Outstanding Paper Honorable Mention; arXiv:2411.19379.

- [36] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KV-Cache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025. Best Paper Award; arXiv:2407.00079.
- [37] Shi Qiu, Yifan Hu, Xintao Wang, Wenhao Zhu, Jianqin Yan, Hao Chen, Kaiqiang Xu, Kai Chen, and Yiming Zhang. Tutti: Making SSD-backed KV cache practical for long-context LLM serving. *arXiv preprint arXiv:2605.03375*, 2026. [PREPRINT] submitted 2026-05-05.
- [38] Ishan Shah, Akanksha Jain, and Calvin Lin. Effective mimicry of Belady’s MIN policy. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.
- [39] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2303.06865.
- [40] Gagandeep Singh, Rakesh Nadig, Jisung Park, Rahul Bera, Nastaran Hajinazar, David Novo, Juan Gómez-Luna, Sander Stuijk, Henk Corporaal, and Onur Mutlu. Sibyl: Adaptive and extensible data placement in hybrid storage systems using online reinforcement learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022. arXiv:2205.07394.
- [41] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 529–544, 2020.
- [42] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.21465.
- [43] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2406.10774.
- [44] Junliang Wang, Jiaqi Hu, Qingping Cao, Yuanrui Zhu, and Xiancheng Lin. Multi-tier dynamic storage of KV cache for LLM inference under resource-constrained conditions. *Complex & Intelligent Systems*, 12(3):104, 2026. Peer-reviewed journal, published online 2026-01-27.
- [45] Yandex Research. KV cache offloading for context-intensive tasks. *arXiv preprint arXiv:2604.08426*, 2026. [PREPRINT]; code at github.com/yandex-research/context-intensive-kv-offloading.
- [46] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. Best Paper Award; arXiv:2405.16444.